
Security Review Report
NM-0755 - 1Inch: Aqua & SwapVM



NETHERMIND
SECURITY

(March 6, 2026)

Contents

1	Executive Summary	2
2	Audited Files	3
2.1	Aqua	3
2.2	SwapVM	3
2.3	solidity-utils	3
3	Summary of Issues	3
4	System Overview	5
4.1	Aqua	5
4.1.1	Core Data Structure	6
4.1.2	Main Functions	6
4.2	SwapVM	6
4.2.1	Core Architecture	6
4.2.2	Execution Context	7
4.2.3	Bytecode Format	7
4.2.4	Order Structure	7
4.2.5	Main Entry Points	7
4.2.6	Balance Management	8
4.2.7	Hook System	8
4.2.8	Instructions Overview	8
4.3	solidity-utils	10
5	Risk Rating Methodology	12
6	Issues	13
6.1	[High] XYCConcentrate liquidity storage gets corrupted when strategy contains more than two tokens	13
6.2	[Medium] Maker's preTransferInHook contributions can reduce taker's payment obligation	14
6.3	[Medium] ship(...) and dock(...) can be called multiple times for the same strategy	15
6.4	[Low] Aqua ship(...) and dock(...) can be called with empty parameters and emit misleading events	16
6.5	[Low] The XYCConcentrate contract may use stale liquidity values when calculating concentrated balances if balances were changed	17
6.6	[Low] The XYCConcentrate contract scaling logic causes price range drift by using incorrect virtual offsets	17
6.7	[Low] XYCConcentrate currentLiquidityPower overflows in multi-token strategies	18
6.8	[Low] XYCConcentrate reverts due to division by zero after first swap if initialLiquidity is zero	19
6.9	[Low] Callldata.slice underflow allows arbitrary hook address redirection	20
6.10	[Low] _decayXD can revert with underflow causing temporary DoS	21
6.11	[Low] _feeAmountIn(...) rounds down favoring taker over maker	22
6.12	[Info] Division by zero if fees are set to 100%	22
6.13	[Info] Extruction targets can cause quote(...) and swap(...) divergence	23
6.14	[Info] Maker can manipulate swap rates via Self-Push in Aqua strategies without balance wrappers	24
6.15	[Best Practices] decayPeriod can be set to zero which invalidates the decay mechanism	25
7	Documentation Evaluation	26
8	Test Suite Evaluation	27
8.1	Tests Output	27
8.1.1	aqua	27
8.1.2	swap-vm	27
8.1.3	solidity-utils	40
8.2	Automated Tools	49
8.2.1	AuditAgent	49
9	About Nethermind	50

1 Executive Summary

This document presents the results of a security review conducted by [Nethermind Security](#) for 1inch's [Aqua](#), [SwapVM](#), and supporting [solidity-utils](#) libraries.

Aqua is a non-custodial shared liquidity layer that tracks virtual balances without taking custody of tokens. This design enables makers (liquidity providers) to back multiple trading strategies with the same capital, as tokens remain in the maker's wallet until a swap is executed.

SwapVM is a Solidity-based virtual machine that executes programmable swap strategies. It supports two order fulfillment modes: EIP-712 signed orders for permissionless execution and Aqua-backed orders that leverage the shared liquidity layer. SwapVM provides a modular instruction set for various AMM curves, fee handling, MEV protection, and swap controls. The architecture supports extensibility through maker hooks and taker callbacks, allowing custom logic at various stages of the swap lifecycle.

The **solidity-utils** repository provides foundational libraries for calldata manipulation, transient storage operations, and simulation utilities used throughout the system.

The audit comprises 1903 lines of the Solidity code. The audit was performed using (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report 15 points of attention, where one is classified as High, two are classified as Medium, eight are classified as Low, three are classified as Informational and one is classified as Best Practices. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the system overview. Section 4 discusses the risk rating methodology. Section 5 details the issues. Section 6 discusses the documentation provided by the client for this audit. Section 7 presents the test suite evaluation and automated tools used. Section 8 concludes the document.

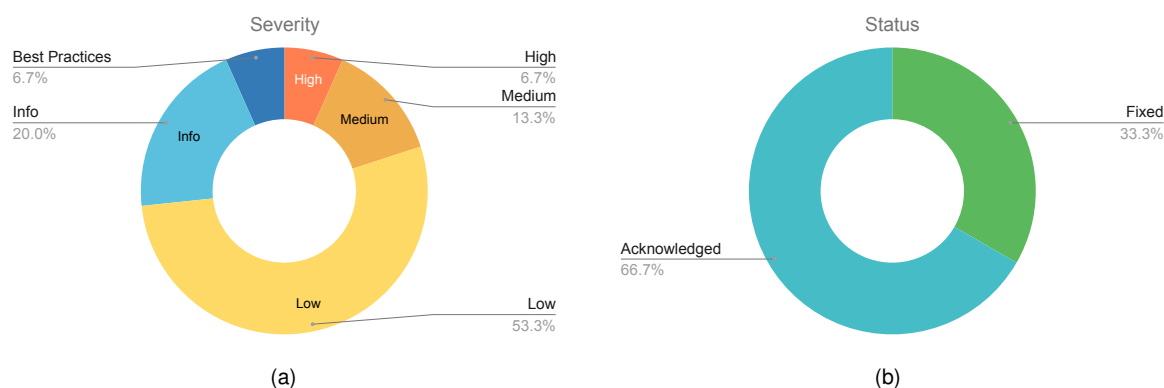


Fig. 1: Distribution of issues: Critical (0), High (1), Medium (2), Low (8), Undetermined (0), Informational (3), Best Practices (1). Distribution of status: Fixed (5), Acknowledged (10), Mitigated (0), Unresolved (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	February 20, 2026
Final Report	March 6, 2026
Initial Commit (aqua)	89cedfa608aa9fdc60d8cd936b76a540e4b9346f
Initial Commit (swap-vm)	e2de56b7b7c759b04060c3b4535f6e489c09a536
Initial Commit (solidity-utils)	38e5f8de705a8821237520ae06f3e9b0d40daf01
Final Commit (aqua)	b51f3b6e3c02fa5208e1429f0f14706b025d9dd6
Final Commit (swap-vm)	b2daef83656b1ccbfcc1c83d3df869c0ebb60badc
Final Commit (solidity-utils)	399012a1128b70c097fd954a336f53d32f4cd3a5
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

2.1 Aqua

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/AquaRouter.sol	5	6	120.0%	4	15
2	src/AquaApp.sol	24	33	137.5%	13	70
3	src/Aqua.sol	61	4	6.6%	16	81
4	src/libs/Balance.sol	20	19	95.0%	4	43
5	src/interfaces/IAqua.sol	23	88	382.6%	16	127
	Total	133	150	112.8%	53	336

2.2 SwapVM

	Contract	LoC	Comments	Ratio	Blank	Total
1	src/SwapVM.sol	223	5	2.2%	36	264
2	src/routers/AquaSwapVMRouter.sol	11	3	27.3%	5	19
3	src/instructions/Balances.sol	108	13	12.0%	23	144
4	src/instructions/PeggedSwap.sol	90	73	81.1%	26	189
5	src/instructions/XCYSwap.sol	24	3	12.5%	7	34
6	src/instructions/Fee.sol	134	29	21.6%	37	200
7	src/instructions/XYCConcentrate.sol	104	30	28.8%	26	160
8	src/instructions/Extraction.sol	62	6	9.7%	8	76
9	src/instructions/Controls.sol	84	26	31.0%	22	132
10	src/instructions/Decay.sol	66	10	15.2%	20	96
11	src/libs/PeggedSwapMath.sol	43	48	111.6%	21	112
12	src/libs/VM.sol	69	26	37.7%	17	112
13	src/libs/MakerTraits.sol	193	4	2.1%	36	233
14	src/libs/TakerTraits.sol	228	3	1.3%	42	273
15	src/opcodes/AquaOpcodes.sol	65	13	20.0%	7	85
16	src/interfaces/ITakerCallbacks.sol	23	3	13.0%	3	29
17	src/interfaces/IMakerHooks.sol	47	5	10.6%	5	57
18	src/interfaces/ISwapVM.sol	24	5	20.8%	6	35
	Total	1598	305	19.1%	347	2250

2.3 solidity-utils

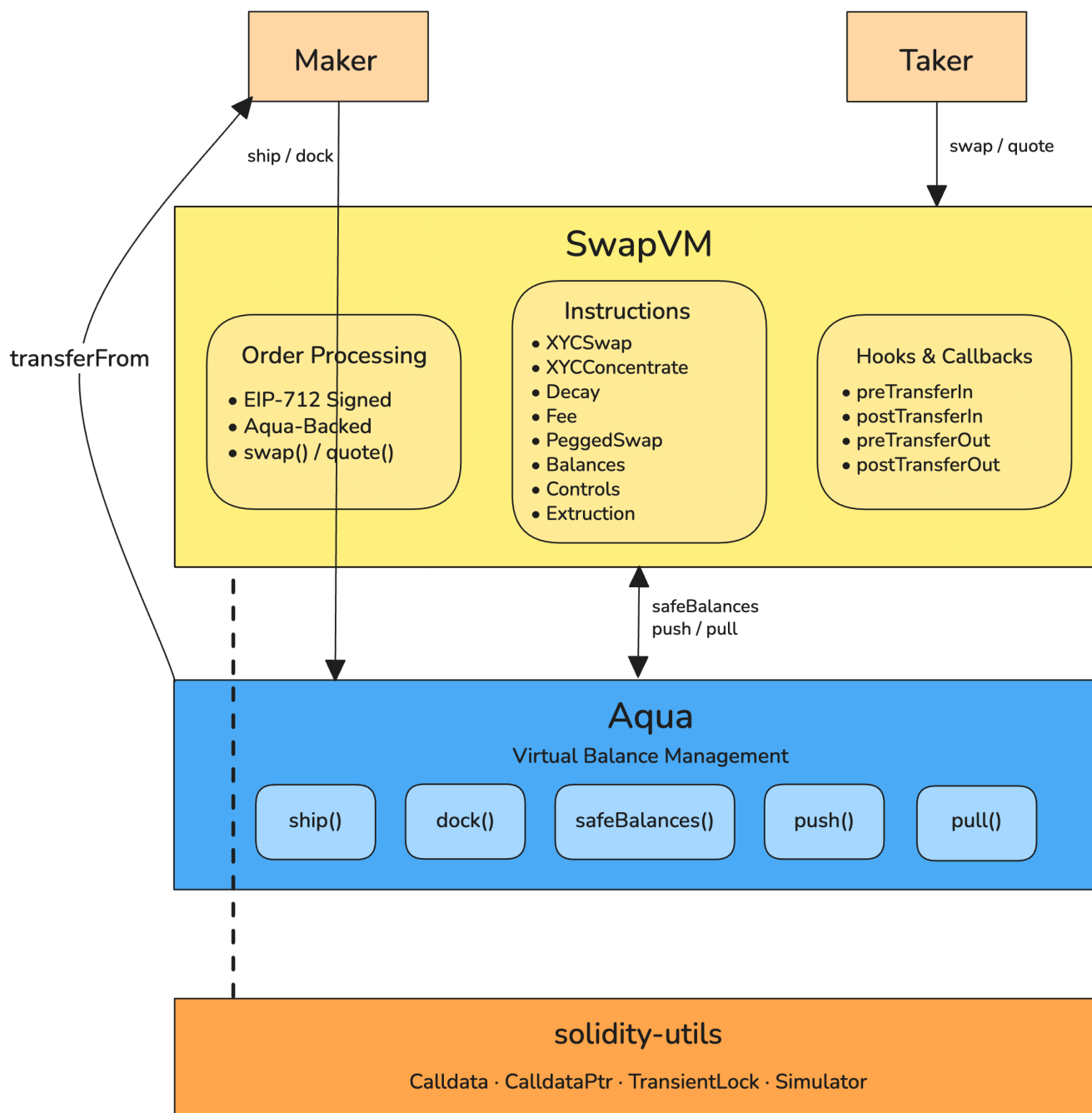
	Contract	LoC	Comments	Ratio	Blank	Total
1	contracts/mixins/Simulator.sol	8	20	250.0%	2	30
2	contracts/libraries/CalldataPtr.sol	16	21	131.2%	4	41
3	contracts/libraries/Transient.sol	88	131	148.9%	21	240
4	contracts/libraries/Calldata.sol	39	38	97.4%	4	81
5	contracts/libraries/TransientLock.sol	21	29	138.1%	9	59
	Total	172	239	139.0%	40	451

3 Summary of Issues

	Finding	Severity	Update
1	XYCConcentrate liquidity storage gets corrupted when strategy contains more than two tokens	High	Fixed
2	Maker's preTransferInHook contributions can reduce taker's payment obligation	Medium	Acknowledged
3	ship(...) and dock(...) can be called multiple times for the same strategy	Medium	Acknowledged
4	Aqua ship(...) and dock(...) can be called with empty parameters and emit misleading events	Low	Acknowledged
5	The XYCConcentrate contract may use stale liquidity values when calculating concentrated balances if balances were changed	Low	Fixed
6	The XYCConcentrate contract scaling logic causes price range drift by using incorrect virtual offsets	Low	Fixed
7	XYCConcentrate currentLiquidityPower overflows in multi-token strategies	Low	Fixed
8	XYCConcentrate reverts due to division by zero after first swap if initialLiquidity is zero	Low	Acknowledged
9	Calldata.slice underflow allows arbitrary hook address redirection	Low	Acknowledged
10	_decayXD can revert with underflow causing temporary DoS	Low	Acknowledged
11	_feeAmountIn(...) rounds down favoring taker over maker	Low	Fixed
12	Division by zero if fees are set to 100%	Info	Acknowledged
13	Extraction targets can cause quote(...) and swap(...) divergence	Info	Acknowledged
14	Maker can manipulate swap rates via Self-Push in Aqua strategies without balance wrappers	Info	Acknowledged
15	decayPeriod can be set to zero which invalidates the decay mechanism	Best Practices	Acknowledged

4 System Overview

The audit scope encompasses three interconnected repositories that together form a modular swap execution framework. **Aqua** provides the non-custodial liquidity layer, **SwapVM** serves as the programmable execution engine, and **solidity-utils** supplies foundational utilities. The diagram below illustrates their interaction.



4.1 Aqua

Aqua is a shared liquidity layer, designed to solve the liquidity fragmentation problem prevalent in traditional DeFi protocols. Instead of locking liquidity into isolated AMM pools, Aqua introduces a registry-based virtual balance system where liquidity providers (LPs) maintain a single token approval while distributing virtual balances across multiple trading strategies simultaneously.

The core innovation is that **tokens never leave the LP's wallet** until a trade is executed. Aqua only tracks virtual balance allocations (allowances), enabling capital efficiency by allowing the same underlying tokens to be allocated to multiple strategies without actual token transfers or redeployment.

4.1.1 Core Data Structure

The contract centers around a 4-dimensional mapping for tracking virtual balances:

```

1 mapping(address maker =>
2   mapping(address app =>
3     mapping(bytes32 strategyHash =>
4       mapping(address token => Balance)))) private _balances;
```

The Balance struct is gas-optimized to fit in a single storage slot (256 bits):

- amount (248 bits): Token balance amount
- tokensCount (8 bits): Number of tokens in strategy, with special value 0xFF indicating a “docked” (deactivated) strategy

4.1.2 Main Functions

- ship() - Creates a new immutable strategy with initial token balances. Computes strategyHash = keccak256(strategy) and stores balances for each token.
- dock() - Deactivates a strategy by setting all token balances to 0 and marking tokensCount as 0xFF (docked). Requires all tokens from the original strategy to be provided
- pull() - Called by apps during swap execution to transfer tokens from a maker's wallet to a recipient. Decrements the virtual balance and performs actual safeTransferFrom.
- push() - Called during swap execution to transfer tokens from the caller to a maker's wallet. Increments the virtual balance. Prevents pushes to inactive/docked strategies.
- rawBalances() - Returns raw balance data (amount + tokensCount) without validation.
- safeBalances() - Returns balances for two tokens with validation that both tokens are part of an active strategy. Reverts if tokens are not in the strategy or the strategy is docked.

Non-Standard ERC20 Considerations: Fee-on-transfer tokens will cause discrepancies between Aqua's virtual balance tracking and actual token flows. When takers pay with fee-on-transfer tokens, makers receive less than the recorded virtual balance increase, causing swap calculations to use inflated values. When makers pay out fee-on-transfer tokens, takers receive less than the calculated amountOut. Additionally, the useTransferFromAndAquaPush payment path will revert entirely, as SwapVM receives fewer tokens than it attempts to push to Aqua.

4.2 SwapVM

SwapVM is a virtual machine for programmable token swaps that executes complex trading strategies from bytecode programs without requiring contract deployment. Instead of deploying separate smart contracts for each trading strategy, makers compose instructions into programs that are signed off-chain and executed on-demand by takers.

The core innovation is that **swap logic is defined through composable bytecode instructions** rather than fixed contract logic. This enables makers to create limit orders, Dutch auctions, AMM-style orders, concentrated liquidity positions, and sophisticated multi-step trading strategies—all without deploying new contracts.

4.2.1 Core Architecture

SwapVM operates on a **4-register model** for computing token swaps.

Note that the information above was revised following the audit. The system now utilizes a **5-register model** and includes a new **amountNetPulled** parameter. The updated struct is shown below.

```

1 struct SwapRegisters {
2   uint256 balanceIn;      // Maker's available input token balance
3   uint256 balanceOut;     // Maker's available output token balance
4   uint256 amountIn;       // Input amount (taker provides OR VM computes)
5   uint256 amountOut;      // Output amount (taker provides OR VM computes)
6   uint256 amountNetPulled; // The net amount pulled from the maker during the swap, used for fee calculations
7 }
```

The execution model follows a simple principle:

- The **taker specifies one amount** (either amountIn or amountOut)
- The **VM computes the other amount** by executing the bytecode program
- Instructions modify the registers to apply pricing logic, fees, and constraints

4.2.2 Execution Context

Every instruction receives a Context structure containing three components:

```

1 struct Context {
2     VM vm; // Execution state (program counter, opcodes)
3     SwapQuery query; // Read-only swap details (maker, taker, tokens)
4     SwapRegisters swap; // Mutable registers for amount computation
5 }

```

The VM struct tracks execution state:

```

1 struct VM {
2     bool isStaticContext; // True for quote(), false for swap()
3     uint256 nextPC; // Program counter for next instruction
4     CalldataPtr programPtr; // Pointer to bytecode program
5     CalldataPtr takerArgsPtr; // Pointer to taker-provided data
6     function(Context memory, bytes calldata) internal[] opcodes;
7 }

```

The SwapQuery struct contains immutable swap information:

```

1 struct SwapQuery {
2     bytes32 orderHash; // Unique order identifier
3     address maker; // Liquidity provider
4     address taker; // Swap executor
5     address tokenIn; // Input token address
6     address tokenOut; // Output token address
7     bool isExactIn; // True if taker specifies input amount
8 }

```

4.2.3 Bytecode Format

Programs are sequences of instructions encoded as:

```
[opcode_index][args_length][args_data]
      1 byte      1 byte      N bytes
```

The VM executes instructions sequentially via the runLoop() function:

```

1 function runLoop(Context memory ctx) internal returns (uint256, uint256) {
2     bytes calldata programBytes = ctx.program();
3     for (uint256 pc = ctx.vm.nextPC; pc < programBytes.length; ) {
4         uint256 opcode = uint8(programBytes[pc++]);
5         uint256 argsLength = uint8(programBytes[pc++]);
6         bytes calldata args = programBytes[pc:pc + argsLength];
7         ctx.vm.nextPC = pc + argsLength;
8         ctx.vm.opcodes[opcode](ctx, args);
9         pc = ctx.vm.nextPC;
10    }
11    return (ctx.swap.amountIn, ctx.swap.amountOut);
12 }

```

4.2.4 Order Structure

Orders are defined by the ISwapVM.Order struct:

```

1 struct Order {
2     address maker; // Liquidity provider address
3     MakerTraits traits; // Packed configuration flags and data offsets
4     bytes data; // Hook data and bytecode program
5 }

```

4.2.5 Main Entry Points

- quote() – Preview function that simulates swap execution in static context (isStaticContext = true). No state changes occur; returns computed amountIn/amountOut.
- swap() – Executes the actual swap. Verifies signature (or Aqua balance), runs the VM, validates results, and performs token transfers.

4.2.6 Balance Management

SwapVM provides flexible balance management to support diverse trading strategies. The balanceIn and balanceOut registers represent the maker's available liquidity and can be populated through three distinct sources, each with different trade-offs.

Static Balances – For trades with fixed exchange rates:

- Balances are provided in the program arguments via `_staticBalancesXD`
- No state persistence between swaps
- The instruction requires both balanceIn and balanceOut to be zero before setting, ensuring balances are not accidentally overwritten

Dynamic Balances – For AMM-style orders with persistent reserves:

- Balances stored in contract storage per orderHash via `_dynamicBalancesXD`
- Automatically updated after each swap based on amountIn and amountOut

Aqua-Sourced Balances – For orders backed by the Aqua shared liquidity protocol:

- When `useAquaInsteadOfSignature` is enabled in `MakerTraits`, balances are loaded from Aqua via `AQUA.safeBalances()` before VM execution begins
- No EIP-712 signature verification is performed; the order hash becomes `keccak256(abi.encode(order))`

Combining Balance Sources: The VM does not enforce mutual exclusivity between balance sources. Program authors must ensure their balance management strategy is internally consistent.

4.2.7 Hook System

SwapVM supports maker hooks at four points during token transfers:

- `preTransferIn` – Before input token transfer
- `postTransferIn` – After input token transfer
- `preTransferOut` – Before output token transfer
- `postTransferOut` – After output token transfer

Hooks are called on contracts implementing `IMakerHooks`. The target can be the maker address or a custom contract specified in the order data.

Additionally, takers can register callbacks via `ITakerCallbacks`:

- `preTransferInCallback` – Taker callback before input transfer
- `preTransferOutCallback` – Taker callback before output transfer

4.2.8 Instructions Overview

Instructions are the building blocks of swap programs. Each instruction is a function that receives the execution context and arguments, and may modify the swap registers, control flow, or consume taker-provided data.

Balance Instructions

`_staticBalancesXD` – Sets balanceIn and balanceOut from provided initial values. Used for fixed-rate orders (limit orders, auctions). Requires both balances to be zero before setting.

```

1 function _staticBalancesXD(Context memory ctx, bytes calldata args) internal pure {
2     require(ctx.swap.balanceIn == 0 && ctx.swap.balanceOut == 0, ...);
3     // Parse tokens and balances from args
4     // Set ctx.swap.balanceIn and ctx.swap.balanceOut for matching tokens
5 }

```

`_dynamicBalancesXD` – Loads existing balances from storage or initializes them. After executing remaining instructions via `runLoop()`, updates stored balances based on swap amounts.

```

1 mapping(bytes32 orderHash => mapping(address token => uint256)) public balances;
2
3 function _dynamicBalancesXD(Context memory ctx, bytes calldata args) internal {
4     // Load or initialize balances
5     (uint256 swapAmountIn, uint256 swapAmountOut) = ctx.runLoop();
6     if (!ctx.vm.isStaticContext) {
7         balances[ctx.query.orderHash][ctx.query.tokenIn] += swapAmountIn;
8         balances[ctx.query.orderHash][ctx.query.tokenOut] -= swapAmountOut;
9     }
10 }

```

Control Flow Instructions

`_salt` – No-op instruction used to make order hashes unique. Accepts arbitrary bytes that affect the program hash but do not modify execution.

`_jump` – Unconditionally sets the program counter to a specified value.

`_jumpIfTokenIn / _jumpIfTokenOut` – Conditional jumps based on whether the input/output token matches a specified address.

`_deadline` – Reverts if `block.timestamp > deadline`. Provides time-limited order validity.

`_onlyTakerTokenBalanceNonZero` – Reverts if the taker holds zero balance of the specified token.

`_onlyTakerTokenBalanceGte` – Reverts if taker's balance of the specified token is below a minimum threshold.

`_onlyTakerTokenSupplyShareGte` – Reverts if taker's balance represents less than a minimum percentage of total supply.

Swap Instructions

`_xycSwapXD` – Implements constant product AMM formula ($x \cdot y = k$). Computes the missing amount based on `isExactIn`:

```

1  function _xycSwapXD(Context memory ctx, bytes calldata) internal pure {
2      require(ctx.swap.balanceIn > 0 && ctx.swap.balanceOut > 0, ...);
3      if (ctx.query.isExactIn) {
4          // amountOut = (amountIn * balanceOut) / (balanceIn + amountIn)
5          ctx.swap.amountOut = (ctx.swap.amountIn * ctx.swap.balanceOut) /
6                               (ctx.swap.balanceIn + ctx.swap.amountIn);
7      } else {
8          // amountIn = ceil((amountOut * balanceIn) / (balanceOut - amountOut))
9          ctx.swap.amountIn = Math.ceilDiv(
10             ctx.swap.amountOut * ctx.swap.balanceIn,
11             ctx.swap.balanceOut - ctx.swap.amountOut
12         );
13     }
14 }
```

`_peggedSwapGrowPriceRange2D` – Square-root linear swap curve optimized for pegged assets (stablecoins, wrapped tokens).

Concentrated Liquidity Instructions

`_xycConcentrateGrowLiquidityXD / _xycConcentrateGrowLiquidity2D` – Scale balances to concentrate liquidity within a price range, similar to Uniswap V3 positions.

```

1  mapping(bytes32 orderHash => uint256) public liquidity;
2
3  function concentratedBalance(bytes32 orderHash, uint256 balance,
4      uint256 delta, uint256 initialLiquidity) public view returns (uint256) {
5      uint256 currentLiquidity = liquidity[orderHash];
6      return currentLiquidity == 0
7         ? balance + delta
8         : balance + delta * currentLiquidity / initialLiquidity;
9  }
```

The instruction adjusts `balanceIn` and `balanceOut` by adding `delta` values that grow proportionally to the liquidity invariant. After `runLoop()` executes the swap, the new liquidity value is stored.

Decay Instruction

`_decayXD` – Implements virtual balance decay. After a swap, virtual offsets are added that decay linearly over a specified period.

```

1  struct DecayingOffset {
2      uint216 offset;
3      uint40 timestamp;
4  }
5
6  mapping(bytes32 => mapping(address => mapping(bool => DecayingOffset))) _offsets;
7
8  function _decayXD(Context memory ctx, bytes calldata args) internal {
9      uint256 period = DecayArgsBuilder.parse(args);
10     // Adjust balances by decayed offsets
11     ctx.swap.balanceIn += _offsets[...][tokenIn][true].getOffset(period);
12     ctx.swap.balanceOut -= _offsets[...][tokenOut][false].getOffset(period);
13
14     (uint256 swapAmountIn, uint256 swapAmountOut) = ctx.runLoop();
15
16     if (!ctx.vm.isStaticContext) {
17         _offsets[...][tokenIn][false].addOffset(swapAmountIn, period);
18         _offsets[...][tokenOut][true].addOffset(swapAmountOut, period);
19     }
```

```
19     }
20 }
```

This creates a virtual price impact that decays over time, discouraging sandwich attacks by making back-running less profitable.

Fee Instructions

`_flatFeeAmountInXD` – Applies a flat percentage fee to the input amount. The fee is deducted before swap computation (for `exactIn`) or added after (for `exactOut`).

```
1 function _feeAmountIn(Context memory ctx, uint256 feeBps) internal returns (uint256) {
2     require(ctx.swap.amountIn == 0 || ctx.swap.amountOut == 0, ...);
3
4     if (ctx.query.isExactIn) {
5         uint256 takerDefinedAmountIn = ctx.swap.amountIn;
6         uint256 feeAmountIn = ctx.swap.amountIn * feeBps / BPS;
7         ctx.swap.amountIn -= feeAmountIn;
8         ctx.runLoop();
9         ctx.swap.amountIn = takerDefinedAmountIn;
10    } else {
11        ctx.runLoop();
12        uint256 feeAmountIn = ctx.swap.amountIn * feeBps / (BPS - feeBps);
13        ctx.swap.amountIn += feeAmountIn;
14    }
15    return feeAmountIn;
16 }
```

`_protocolFeeAmountInXD` / `_aquaProtocolFeeAmountInXD` – Apply a fee and transfer it to a specified recipient. The Aqua variant uses `AQUA.pull()` instead of `direct transferFrom`.

`_dynamicProtocolFeeAmountInXD` / `_aquaDynamicProtocolFeeAmountInXD` – Query an external `IProtocolFeeProvider` contract via `staticcall` to determine fee amount and recipient dynamically.

Extraction Instruction

`_extraction` – Enables external contract calls for custom logic. The target contract must implement `IExtraction` or `IStaticExtraction`:

```
1 interface IExtraction {
2     function extraction(
3         bool isStaticContext,
4         uint256 nextPC,
5         SwapQuery calldata query,
6         SwapRegisters calldata swap,
7         bytes calldata args,
8         bytes calldata takerData
9     ) external returns (
10        uint256 updatedNextPC,
11        uint256 choppedLength,
12        SwapRegisters memory updatedSwap
13    );
14 }
```

The external contract can:

- Modify swap registers (`updatedSwap`)
- Change the program counter (`updatedNextPC`)
- Consume taker data (`choppedLength` bytes)

Security Consideration: The `_extraction` instruction represents the most security-critical opcode in SwapVM due to its ability to delegate execution to arbitrary external contracts. This poses significant risks, orders containing `_extraction` opcodes should be treated with extreme caution by takers, and the target contract should be thoroughly audited before any maker signs such an order.

4.3 solidity-utils

solidity-utils is a collection of utility libraries and contracts created to handle **calldata manipulation**, and **transient storage (EIP-1153)**.

Calldata Library

Provides slicing of calldata byte arrays without memory copying. Operations work directly on calldata offsets:

```
1 library Calldata {
2     function slice(bytes calldata calls, uint256 begin, uint256 end)
3         internal pure returns (bytes calldata res);
4     function slice(bytes calldata calls, uint256 begin, uint256 end, bytes4 exception)
```

```

5     internal pure returns (bytes calldata res);
6     function slice(bytes calldata calls, uint256 begin)
7         internal pure returns (bytes calldata res);
8     function slice(bytes calldata calls, uint256 begin, bytes4 exception)
9         internal pure returns (bytes calldata res);
10 }

```

CalldataPtr Library

User-defined value type representing a packed calldata pointer:

```

1 type CalldataPtr is uint256;
2
3 library CalldataPtrLib {
4     function from(bytes calldata data) internal pure returns (CalldataPtr ptr);
5     function toBytes(CalldataPtr ptr) internal pure returns (bytes calldata data);
6 }

```

The CalldataPtr packs offset and length into a single uint256:

- Upper 128 bits: calldata offset
- Lower 128 bits: data length

Transient Library

Provides drop-in replacement types for uint256, address, and bytes32 with transient storage semantics:

```

1 struct tuint256 { uint256 _raw; }
2 struct taddress { address _raw; }
3 struct tbytes32 { bytes32 _raw; }

```

The TransientLib library provides operations for these types:

- tload() / tstore() - Basic load and store operations using EIP-1153 opcodes
- inc() / dec() - Atomic increment/decrement with overflow/underflow checks
- unsafeInc() / unsafeDec() - Unchecked variants for gas savings when safety is guaranteed
- initAndAdd() - Initialize with default value if zero, then add

TransientLock Library

Gas-efficient reentrancy protection using transient storage:

```

1 struct TransientLock {
2     tuint256 _raw;
3 }
4
5 library TransientLockLib {
6     function lock(TransientLock storage self) internal;
7     function unlock(TransientLock storage self) internal;
8     function isLocked(TransientLock storage self) internal view returns (bool);
9 }

```

- lock() - Acquires lock by incrementing counter; reverts with UnexpectedLock() if already locked
- unlock() - Releases lock by decrementing counter; reverts with UnexpectedUnlock() if not locked
- isLocked() - View function to check current lock state

Simulator Contract

A utility contract for off-chain transaction simulation:

```

1 contract Simulator {
2     error Simulated(address delegatee, bytes data, bool success, bytes result);
3
4     function simulate(address delegatee, bytes calldata data) external payable;
5 }

```

- simulate() - Executes a delegatecall to the target address and always reverts with the Simulated error containing the execution result
- Enables off-chain preview of transaction effects without persisting state changes
- Returns delegatee address, calldata, success status, and return data/revert reason in the error payload

5 Risk Rating Methodology

The risk rating methodology used by [Nethermind Security](#) follows the principles established by the [OWASP Foundation](#). The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

- a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;
- b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;
- c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

- a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;
- b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;
- c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	Medium	High
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, [Nethermind Security](#) also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

- a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;
- b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;
- c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [High] XYCConcentrate liquidity storage gets corrupted when strategy contains more than two tokens

File(s): swap-vm/src/instructions/XYCConcentrate.sol

Description: The XYCConcentrate instruction uses a single liquidity value per orderHash to track the concentrated liquidity invariant:

```

1 // XYCConcentrate.sol
2 mapping(bytes32 orderHash => uint256) public liquidity;
3
4 function concentratedBalance(bytes32 orderHash, uint256 balance, uint256 delta, uint256 initialLiquidity) public view
5     → returns (uint256) {
6     uint256 currentLiquidity = liquidity[orderHash];
7     return currentLiquidity == 0 ? balance + delta : balance + delta * currentLiquidity / initialLiquidity;
8 }
9
10 function _updateScales(Context memory ctx) private {
11     // ...
12     if (!ctx.vm.isStaticContext) {
13         uint256 newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut);
14         liquidity[ctx.query.orderHash] = Math.sqrt(newInv);
15     }
16 }

```

This design assumes each orderHash uniquely identifies a single token pair. However, XYCConcentrate is fundamentally designed for exactly 2 tokens, and the protocol does not enforce this constraint.

The primary issue occurs when 3 or more tokens are shipped in a single call:

```

1 ship(app, strategy, [tokenA, tokenB, tokenC], [amount1, amount2, amount3])

```

Since XYCConcentrate expects exactly 2 tokens per strategy, swaps on any pair within this set (tokenA/tokenB, tokenA/tokenC, or tokenB/tokenC) will all share the same liquidity[orderHash] storage. Each swap calculates liquidity based on its own token pair's reserves but writes to the shared storage slot, corrupting calculations for the other pairs.

1. A swap on tokenA/tokenB calculates and stores liquidity[orderHash] based on tokenA/tokenB reserves;
2. A subsequent swap on tokenA/tokenC reads this liquidity value, which is incorrect for tokenA/tokenC;
3. The concentratedBalance() calculation uses the wrong scaling factor: balance + delta * wrongLiquidity / initialLiquidity;
4. Swap outputs are incorrect;
5. After the tokenA/tokenC swap, liquidity[orderHash] is overwritten with tokenA/tokenC's calculated value;
6. Future tokenA/tokenB swaps now have corrupted liquidity tracking;
7. This bidirectional corruption continues indefinitely;

A secondary collision scenario can occur when shipping the same strategy twice with different token sets. Normally, makers should use the _salt instruction from Controls.sol to ensure unique orderHash values for different strategies:

```

1 /// @dev This instruction does nothing and can be used for uniqueness order hash value.
2 function _salt(Context memory /* ctx */, bytes calldata /* args */) internal pure { }

```

However, if a maker forgets to include a unique salt, strategies with identical parameters will produce identical orderHash values. Since build2D() does not embed token addresses:

```

1 function build2D(address tokenA, address tokenB, uint256 deltaA, uint256 deltaB, uint256 liquidity) internal pure
2     → returns (bytes memory) {
3     (uint256 deltaLt, uint256 deltaGt) = tokenA < tokenB ? (deltaA, deltaB) : (deltaB, deltaA);
4     return abi.encodePacked(deltaLt, deltaGt, liquidity);
5 }

```

A maker could ship the same strategy bytes with different tokens:

- ship(swapVM, strategy, [tokenA, tokenB], amounts1);
- ship(swapVM, strategy, [tokenC, tokenD], amounts2);

Both calls use the same strategy bytes (same orderHash), so Aqua stores balances separately per token, but XYCConcentrate shares liquidity[orderHash] between them, causing the same corruption described above.

Recommendation(s): Consider restricting the instruction to only work with two tokens.

Status: Fixed.

Update from the client: Fixed in [PR #82](#)

Update from Nethermind: The initially introduced changes in [PR #58](#) fixed the highlighted issue. However, it introduced the concern XYCConcentrate currentLiquidityPower overflows in multi-token strategies and the new version for three and four tokens also presents the problems highlighted in The XYCConcentrate contract scaling logic causes price range drift by using incorrect virtual offsets and The XYCConcentrate contract may use stale liquidity values when calculating concentrated balances if balances were changed. Afterwards, the issue was fixed with [PR #82](#) that restricted the strategy to only work with two tokens.

6.2 [Medium] Maker's preTransferInHook contributions can reduce taker's payment obligation

File(s): [swap-vm/src/SwapVM.sol](#)

Description: In the swap(...) function, the originalAquaBalanceIn is captured before any hooks execute:

```

1 // SwapVM.sol swap(...)
2 // ...
3 uint256 originalAquaBalanceIn = ctx.swap.balanceIn; // Snapshot before hooks
4 (amountIn, amountOut) = ctx.runLoop();
5 // ...
6 _transferIn(ctx, order, takerTraits, takerData, originalAquaBalanceIn);
7 // ...

```

Within _transferIn(...), the maker's preTransferInHook executes before the taker's callback, and the balance validation uses the early snapshot:

```

1 // SwapVM.sol
2 function _transferIn(..., uint256 originalAquaBalanceIn) private {
3     if (order.traits.hasPreTransferInHook()) {
4         // @audit Maker's hook runs first
5         target.preTransferIn(...);
6     }
7
8     if (takerTraits.hasPreTransferInCallback()) {
9         // @audit Taker's callback runs second
10        ITakerCallbacks(ctx.query.taker).preTransferInCallback(...);
11    }
12
13    // ...
14    (uint256 balanceIn,) = AQUA.rawBalances(...);
15    require(balanceIn >= originalAquaBalanceIn + ctx.swap.amountIn, ...); // Uses early snapshot
16 }

```

If the maker's preTransferInHook pushes tokens to Aqua (e.g., depositing yield or topping off balances), the taker can exploit this by paying fewer tokens. As an example scenario:

1. originalAquaBalanceIn = 100 (captured before hooks);
2. amountIn = 50 (required taker payment);
3. Maker's hook pushes 20 tokens → Aqua balance = 120;
4. Taker only pushes 30 tokens → Aqua balance = 150;
5. Check: 150 >= 100 + 50 passes;

The taker effectively paid 30 instead of 50, with the maker's hook contribution covering the difference.

Recommendation(s): Consider documenting that makers should use postTransferInHook instead of preTransferInHook when adding tokens to avoid this behavior.

Status: Acknowledged.

Update from the client: This scenario is indeed possible, and the maker needs to take this possibility into account when designing a strategy. We will add a note about this to the documentation.

6.3 [Medium] ship(...) and dock(...) can be called multiple times for the same strategy

File(s): aqua/src/Aqua.sol

Description: The ship(...) function allows a maker to ship multiple disjoint token sets to the same strategyHash because the immutability check is performed per-token rather than per-strategy:

```

1 // Aqua.sol
2 function ship(address app, bytes calldata strategy, address[] calldata tokens, uint256[] calldata amounts) external
3   returns(bytes32 strategyHash) {
4     strategyHash = keccak256(strategy);
5     uint8 tokensCount = tokens.length.toUint8();
6     // ...
7     for (uint256 i = 0; i < tokens.length; i++) {
8       Balance storage balance = _balances[msg.sender][app][strategyHash][tokens[i]];
9       require(balance.tokensCount == 0, StrategiesMustBeImmutable(app, strategyHash)); // @audit Per-token check
10      balance.store(amounts[i].toUint248(), tokensCount);
11      // ...
12    }
13  }

```

This allows shipping [tokenA, tokenB] followed by [tokenC, tokenD] to the same strategy, as long as the token sets are disjoint.

When the maker attempts to dock the strategy, the dock() function enforces that tokensCount == tokens.length:

```

1 // Aqua.sol - dock()
2 function dock(address app, bytes32 strategyHash, address[] calldata tokens) external {
3   for (uint256 i = 0; i < tokens.length; i++) {
4     Balance storage balance = _balances[msg.sender][app][strategyHash][tokens[i]];
5     require(balance.tokensCount == tokens.length, DockingShouldCloseAllTokens(app, strategyHash));
6     balance.store(0, _DOCKED);
7   }
8   emit Docked(msg.sender, app, strategyHash);
9 }

```

This creates a problematic scenario:

1. Maker ships [tokenA, tokenB] → both have tokensCount = 2;
2. Maker ships [tokenC, tokenD] → both have tokensCount = 2;
3. Maker calls dock([tokenA, tokenB]) → succeeds because 2 == 2;
4. tokenC and tokenD remain active with tokensCount = 2;
5. Maker cannot dock all four tokens at once: dock([A,B,C,D]) reverts because tokensCount(2) != tokens.length(4);

The DockingShouldCloseAllTokens error name is misleading, it suggests the invariant enforces closing all tokens in the strategy, but it only verifies the batch size matches.

Additionally, the Docked event does not emit which tokens were docked:

```

1 emit Docked(msg.sender, app, strategyHash);

```

This makes it hard for off-chain systems to distinguish between partial and full docking and active strategies might be perceived as inactive by off-chain systems.

Recommendation(s): Consider adding validation to prevent shipping the same strategy multiple times.

Status: Acknowledged.

Update from the client: This works by design; we do not guarantee full correspondence between ship and dock, as that would require using storage and would make protocol usage significantly more expensive.

6.4 [Low] Aqua ship(...) and dock(...) can be called with empty parameters and emit misleading events

File(s): `aqua/src/Aqua.sol`

Description: The `ship()` and `dock()` functions in `Aqua.sol` can be called with empty tokens arrays (and in the case of `ship()`, an empty strategy), resulting in events being emitted without any meaningful state changes. This can pollute event logs and potentially mislead off-chain systems that rely on these events for indexing or monitoring.

```

1 // src/Aqua.sol
2
3 function ship(address app, bytes calldata strategy, address[] calldata tokens, uint256[] calldata amounts) external
4 → returns(bytes32 strategyHash) {
5     strategyHash = keccak256(strategy); // @audit keccak256("") is valid
6     uint8 tokensCount = tokens.length.toUint8(); // @audit tokensCount = 0 if empty
7     require(tokensCount != _DOCKED, MaxNumberOfTokensExceeded(tokensCount, _DOCKED)); // @audit 0 != 255, passes
8
9     emit Shipped(msg.sender, app, strategyHash, strategy); // @audit Event emitted unconditionally
10    for (uint256 i = 0; i < tokens.length; i++) { // @audit Loop never executes if tokens.length == 0
11        Balance storage balance = _balances[msg.sender][app][strategyHash][tokens[i]];
12        require(balance.tokensCount == 0, StrategiesMustBeImmutable(app, strategyHash));
13        balance.store(amounts[i].toUint248(), tokensCount);
14        emit Pushed(msg.sender, app, strategyHash, tokens[i], amounts[i]);
15    }
16 }

```

When called with `tokens = []` and `strategy = ""`:

1. `tokensCount = 0`, passes the `!= _DOCKED` check;
2. Shipped event is emitted with the empty strategy;
3. The for loop never executes (no balances stored, no Pushed events);
4. A Shipped event is emitted for a strategy with zero tokens and no actual liquidity;

```

1 // src/Aqua.sol
2
3 function dock(address app, bytes32 strategyHash, address[] calldata tokens) external {
4     for (uint256 i = 0; i < tokens.length; i++) { // @audit Loop never executes if tokens.length == 0
5         Balance storage balance = _balances[msg.sender][app][strategyHash][tokens[i]];
6         require(balance.tokensCount == tokens.length, DockingShouldCloseAllTokens(app, strategyHash));
7         balance.store(0, _DOCKED);
8     }
9     emit Docked(msg.sender, app, strategyHash); // @audit Event emitted unconditionally
10 }

```

When called with `tokens = []` and any `strategyHash`:

1. The for loop never executes;
2. No validation that `strategyHash` corresponds to an existing strategy;
3. Docked event is emitted regardless;
4. A Docked event can be emitted for:
 - A non-existent strategy
 - A strategy that was never shipped
 - Any arbitrary `strategyHash` value

Recommendation(s): Consider adding validations to prevent these scenarios.

Status: Acknowledged.

6.5 [Low] The XYCConcentrate contract may use stale liquidity values when calculating concentrated balances if balances were changed

File(s): [src/instructions/XYCConcentrate.sol](#)

Description: The XYCConcentrate contract is designed to provide liquidity concentration within specific price bounds for the XYCSwap formula. To maintain the correct concentration ratio across multiple trades, the contract tracks a liquidity value for each orderHash, which represents the square root of the constant product invariant after the previous swap.

When a new swap is initiated, the concentratedBalance(...) function uses the stored liquidity to scale the concentration delta before the swap execution begins. However, if the underlying token balances for the orderHash have changed since the last swap (for example, if a maker added more liquidity to the strategy), the stored liquidity value becomes stale. The contract only updates this value in the _updateScales(...) function, which is called after the current swap execution has already completed.

```

1 function concentratedBalance(bytes32 orderHash, uint256 balance, uint256 delta, uint256 initialLiquidity) public view
2   → returns (uint256) {
3     // @audit-issue The liquidity[orderHash] reflects the state of the pool after the LAST swap.
4     uint256 currentLiquidity = liquidity[orderHash];
5     // @audit-issue If the 'balance' parameter has increased externally, 'currentLiquidity' is stale.
6     return currentLiquidity == 0 ? balance + delta : balance + delta * currentLiquidity / initialLiquidity;
7   }

```

This means that any trade occurring after a balance modification will be executed on a curve that does not accurately reflect the expected depth. The delta will be scaled incorrectly relative to the new balances, leading to trades happening with incorrect price bounds.

Recommendation(s): Consider recalculating the liquidity based on the actual current balances at the start of the XYCConcentrate instruction, rather than relying on a value stored at the end of the previous trade.

Status: Fixed.

Update from the client: The described scenario seems to be possible only for AquaOpcodes and not possible for Opcodes that use dynamicBalances. Even there, however, such a case looks like ignoring our strategy immutability requirement. We will add this to the documentation to warn makers against such actions.

Update from the client: Fixed in [PR #82](#).

6.6 [Low] The XYCConcentrate contract scaling logic causes price range drift by using incorrect virtual offsets

File(s): [src/instructions/XYCConcentrate.sol](#)

Description: The XYCConcentrate contract implements a liquidity concentration mechanism for the XYCSwap formula. It aims to simulate concentrated liquidity by adding virtual offsets (delta) to the real token balances. To maintain the concentration ratio as the pool's invariant (L) may change through trades, the contract scales the initial delta values using the ratio between current liquidity and initialLiquidity.

The mathematical model used for the concentrated curve is $L = \sqrt{(x + \Delta x) \cdot (y + \Delta y)}$, where x and y are real reserves and $\Delta x, \Delta y$ are the virtual offsets.

```

1 function concentratedBalance(bytes32 orderHash, uint256 balance, uint256 delta, uint256 initialLiquidity)
2   public view returns (uint256) {
3     uint256 currentLiquidity = liquidity[orderHash];
4     // @audit-issue This scales deltas linearly with L, but updating deltas changes L itself.
5     return currentLiquidity == 0 ? balance + delta : balance + delta * currentLiquidity / initialLiquidity;
6   }
7

```

The problem is that the currentLiquidity used to scale the delta for the *current* swap was calculated at the end of the *previous* swap using the *previous* swap's scaled offsets. When these new scaled offsets are applied to the current real balances, they define a new curve with a different invariant $L_{new} = \sqrt{(x_{real} + \Delta x_{scaled}) \cdot (y_{real} + \Delta y_{scaled})}$.

Because the scaling factor is applied to the offsets that help define the invariant, the resulting L_{new} does not stay consistent with the L used for scaling. This mismatch means the swap is executed on a curve where the virtual offsets no longer correspond to the intended price bounds established during the initial setup.

Recommendation(s): Consider revisiting the scaling model to ensure that the virtual offsets are calculated as a function of the constant price bounds and the current invariant.

Status: Fixed.

Update from the client: We are aware of this characteristic and will further research the economic feasibility and risks of using such instructions.

Update from the client: Fixed in [PR #82](#).

6.7 [Low] XYCConcentrate currentLiquidityPower overflows in multi-token strategies

File(s): [src/instructions/XYCConcentrate.sol](#)

Description: The `_parseXD` function computes the product of all N concentrated balances into a single `uint256` before taking the N-th root to derive the current liquidity:

```

1  function _parseXD(Context memory ctx, bytes calldata args) internal view returns (uint256 currentLiquidityPower,
2  ↪ uint256 initialLiquidityRoot, uint256 deltaIn, uint256 deltaOut) {
3      // ...
4      currentLiquidityPower = 1;
5
6      for (uint256 i = 0; i < tokensCount; i++) {
7          address token = address(bytes20(tokens.slice(i * 20)));
8          uint256 balance = concentratedBalances[ctx.query.orderHash][token];
9          if (balance == 0) {
10             balance = uint128(bytes16(initialConcentrated.slice(i * 16)));
11         }
12         currentLiquidityPower *= balance;
13         // ...
14     }
15 }

```

This `currentLiquidityPower` is then passed to `CubeRoot.cbrt()` (3D) or `Math.sqrt(Math.sqrt())` (4D) in the calling functions.

- For 3D pools (`_xycConcentrateGrowLiquidity3D`): three concentrated balances each above approximately 49M tokens (at 18 decimals, i.e., 4.9×10^{25}) produce a product that overflows `uint256`;
- For 4D pools (`_xycConcentrateGrowLiquidity4D`): four concentrated balances each above approximately 18 tokens (at 18 decimals, i.e., 1.8×10^{19}) produce a product that overflows `uint256`;

Even a pool that initially operates within safe bounds can overflow after fee accumulation. Each swap updates the concentrated balances in `_updateConcentratedBalancesXD`:

```

1  function _updateConcentratedBalancesXD(Context memory ctx) internal {
2      if (!ctx.vm.isStaticContext) {
3          concentratedBalances[ctx.query.orderHash][ctx.query.tokenIn] = ctx.swap.balanceIn + ctx.swap.amountIn;
4          concentratedBalances[ctx.query.orderHash][ctx.query.tokenOut] = ctx.swap.balanceOut - ctx.swap.amountOut;
5      }
6  }

```

When fees are collected, the incoming token's concentrated balance grows over time (the maker retains a portion of `amountIn` as profit, inflating `balanceIn + amountIn` across swaps). A pool that starts safely below the overflow boundary will gradually grow as fees accumulate, eventually crossing the threshold. At that point, the strategy bricks permanently since all subsequent swaps revert and the stored `concentratedBalances` cannot be reduced.

Recommendation(s): Consider the new approach used and limiting the types of tokens can be used to not overflow

Status: Fixed.

Update from the client: Fixed in [PR #82](#).

6.8 [Low] XYCConcentrate reverts due to division by zero after first swap if initialLiquidity is zero

File(s): swap-vm/src/instructions/XYCConcentrate.sol

Description: The concentratedBalance(...) function in XYCConcentrate has two code paths based on whether currentLiquidity (stored in liquidity[orderHash]) is zero:

```

1 // XYCConcentrate.sol
2 function concentratedBalance(bytes32 orderHash, uint256 balance, uint256 delta, uint256 initialLiquidity) public view
3   returns (uint256) {
4     uint256 currentLiquidity = liquidity[orderHash];
5     return currentLiquidity == 0 ? balance + delta : balance + delta * currentLiquidity / initialLiquidity;
  }
```

The initialLiquidity parameter is encoded in the strategy bytecode when the maker builds the program. If initialLiquidity is set to zero:

1. liquidity[orderHash] starts at 0, so the function takes the first path (balance + delta) which doesn't use initialLiquidity;
2. _updateScales(...) sets non-zero liquidity. After the swap, liquidity is updated:

```

1 function _updateScales(Context memory ctx) private {
2   // ...
3   uint256 newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut);
4   liquidity[ctx.query.orderHash] = Math.sqrt(newInv); // @audit Now non-zero
5 }
```

3. Second swap reverts. currentLiquidity != 0, so the function takes the second path which divides by initialLiquidity:

```

1 balance + delta * currentLiquidity / initialLiquidity // @audit Division by zero
```

This results in a Solidity panic with no descriptive error message.

The strategy becomes permanently bricked, all subsequent swaps will revert, causing a denial of service. The maker cannot recover the strategy; they must dock it and create a new one.

Recommendation(s): Consider adding a validation check to prevent zero initialLiquidity.

Status: Acknowledged.

Update from the client: A strategy that uses the XYCConcentrate instruction while having initialLiquidity equal to 0 is considered invalid.

6.9 [Low] Calldata.slice underflow allows arbitrary hook address redirection

File(s): solidity-utils/contracts/libraries/Calldata.sol

Description: The Calldata.slice() function contains a vulnerability where it fails to validate that begin <= end before computing the slice length in assembly. When begin > end, the length calculation sub(end, begin) underflows to an extremely large value, causing the function to return a corrupted slice that reads from an unintended memory region.

```

1 // contracts/libraries/Calldata.sol
2
3 function slice(bytes calldata calls, uint256 begin, uint256 end, bytes4 exception) internal pure returns (bytes calldata
4   ↪ res) {
5     // @audit Only checks that end doesn't exceed data length
6     // @audit Missing: require(begin <= end)
7     if (end > calls.length) {
8       _revert(exception);
9     }
10    assembly ("memory-safe") {
11      res.offset := add(calls.offset, begin)
12      // @audit Underflows if begin > end
13      res.length := sub(end, begin)
14    }
15  }

```

This vulnerability is exploitable through the MakerTraits and TakerTraits libraries, which use Calldata.slice() to extract hook data from order data. A malicious maker can craft an order with inverted indices in the traits field, causing the slice to read from an unintended offset in the order data.

The MakerTraitsLib.build() function normally creates cumulative indices that are strictly increasing (e.g., index0=31, index1=63). However, a maker can manually construct traits with inverted indices (e.g., index0=63, index1=31), which are then passed to _getDataSlice():

```

1 // src/libs/MakerTraits.sol
2
3 function _getDataSlice(MakerTraits traits, bytes calldata data, OrderDataSlices slice) private pure returns (bytes
4   ↪ calldata) {
5     uint256 packed = (MakerTraits.unwrap(traits) >> _ORDER_DATA_SLICES_INDEXES_BIT_OFFSET) & type(uint64).max;
6     uint256 startOffset = slice == OrderDataSlices.PreTransferInHook ? 0 : uint16(packed >> (uint8(slice) - 1) * 16);
7     uint256 stopOffset = uint16(packed >> uint8(slice) * 16);
8     // @audit If indices are inverted, startOffset > stopOffset → underflow
9     return data.slice(startOffset, stopOffset, _SLICE_ERROR_SELECTORS[uint8(slice)]);
10  }

```

When the hook address is extracted from the corrupted slice via _getDataSliceWithTarget(), it reads 20 bytes from the wrong offset in the order data - typically from the program bytecode section. This results in the hook call being directed to an unintended address, which could be:

1. A non-contract address (causes revert);
2. Any contract embedded in the program including token contracts;
3. A maker-controlled contract deliberately embedded in the order data;

An example attack scenario is as follows:

1. Maker creates an order with legitimate-looking hooks and valid program logic;
2. Maker manually crafts traits with inverted indices instead of using MakerTraitsLib.build();
3. Taker calls quote() to simulate the swap → **succeeds** (hooks are not executed in static context);
4. Taker sees valid quote amounts and proceeds to call swap();
5. During swap(), the hook parsing reads from the wrong offset due to underflow;
6. The hook call goes to an arbitrary address derived from program bytecode;
7. Call fails with "call to non-contract address" or unexpected behavior;
8. Taker loses gas fees; swap fails despite valid quote and valid hooks;

Recommendation(s): Consider adding a check to prevent this scenario where begin > end.

Status: Acknowledged.

Update from the client: We want to keep the library as gas-cheap as possible, so we do not plan to add additional checks.

6.10 [Low] _decayXD can revert with underflow causing temporary DoS

File(s): swap-vm/src/instructions/Decay.sol

Description: The _decayXD(...) instruction adjusts virtual balances using decaying offsets from previous swaps to provide MEV protection:

```
1 // Decay.sol
2 function _decayXD(Context memory ctx, bytes calldata args) internal {
3     require(ctx.swap.amountIn == 0 || ctx.swap.amountOut == 0, DecayShouldBeCalledBeforeSwapAmountsComputation(...));
4
5     uint256 period = DecayArgsBuilder.parse(args);
6     ctx.swap.balanceIn += _offsets[ctx.query.orderHash][ctx.query.tokenIn][true].getOffset(period);
7     ctx.swap.balanceOut -= _offsets[ctx.query.orderHash][ctx.query.tokenOut][false].getOffset(period); // @audit
8     ↳ Underflow possible
9
10    (uint256 swapAmountIn, uint256 swapAmountOut) = ctx.runLoop();
11
12    if (!ctx.vm.isStaticContext) {
13        _offsets[ctx.query.orderHash][ctx.query.tokenIn][false].addOffset(swapAmountIn, period);
14        _offsets[ctx.query.orderHash][ctx.query.tokenOut][true].addOffset(swapAmountOut, period);
15    }
16 }
```

The decay mechanism uses cross-directional keying: offsets stored from A→B swaps affect subsequent B→A swaps, and vice versa. If the offset for tokenOut[false] exceeds ctx.swap.balanceOut, the subtraction underflows and reverts. This case is reachable when _decayXD(...) is used with static balances.

The addOffset(...) function adds to the current decayed offset and resets the timestamp:

```
1 function addOffset(DecayingOffset storage self, uint256 offset, uint256 decayPeriod) internal {
2     _store(self, (getOffset(self, decayPeriod) + offset).toUint216(), uint40(block.timestamp));
3 }
```

Multiple same-direction swaps in quick succession accumulate offsets. After several same-direction swaps, the accumulated offset can exceed the pool's balance for that token.

An attacker could theoretically prolong the DoS by making small swaps in the same direction to reset the decay timer:

```
1 // @audit addOffset resets timestamp to block.timestamp
2 _store(self, (getOffset(self, decayPeriod) + offset).toUint216(), uint40(block.timestamp));
```

Each small swap:

- Adds a small amount to the already-decayed offset;
- Resets the decay timer to current time;
- Extends the time until the offset fully decays;

The issue is largely self-correcting, once the decay period passes without same-direction swaps, offsets return to zero and normal operation resumes.

Recommendation(s): Consider implementing checks to prevent underflow or revert with a descriptive error.

Status: Acknowledged.

Update from the client: The maker must validate strategies before use as static balances are not compatible with Decay instruction.

6.11 [Low] `_feeAmountIn(...)` rounds down favoring taker over maker

File(s): `swap-vm/src/instructions/Fee.sol`

Description: The `_feeAmountIn(...)` function calculates fees using Solidity's default floor division, which rounds down:

```

1 // Fee.sol - _feeAmountIn()
2 function _feeAmountIn(Context memory ctx, uint256 feeBps) internal returns (uint256 feeAmountIn) {
3     require(ctx.swap.amountIn == 0 || ctx.swap.amountOut == 0, FeeShouldBeAppliedBeforeSwapAmountsComputation());
4
5     if (ctx.query.isExactIn) {
6         uint256 takerDefinedAmountIn = ctx.swap.amountIn;
7         feeAmountIn = ctx.swap.amountIn * feeBps / BPS; // @audit Rounds down
8         ctx.swap.amountIn -= feeAmountIn;
9         ctx.runLoop();
10        ctx.swap.amountIn = takerDefinedAmountIn;
11    } else {
12        ctx.runLoop();
13        feeAmountIn = ctx.swap.amountIn * feeBps / (BPS - feeBps); // @audit Rounds down
14        ctx.swap.amountIn += feeAmountIn;
15    }
16 }
```

Rounding down favors the taker (who pays less fees) at the expense of the maker (who collects less fees).

For low decimal, high value tokens like WBTC (8 decimals), small trade amounts can result in significant loss of fees. On low gas cost chains, a taker can exploit this by splitting large trades into many micro-swaps below the fee threshold to avoid paying fees.

Recommendation(s): Consider rounding fee calculations up instead of down to favor the maker.

Status: Fixed.

Update from the client: Fixed in [PR #73](#)

6.12 [Info] Division by zero if fees are set to 100%

File(s): `swap-vm/src/instructions/Fee.sol`

Description: The `_feeAmountIn(...)` function calculates fee amounts for swaps. When `isExactIn` is false (exact output mode), the fee is calculated using the formula `ctx.swap.amountIn * feeBps / (BPS - feeBps)`:

```

1 // Fee.sol
2 function _feeAmountIn(Context memory ctx, uint256 feeBps) internal returns (uint256 feeAmountIn) {
3     require(ctx.swap.amountIn == 0 || ctx.swap.amountOut == 0, FeeShouldBeAppliedBeforeSwapAmountsComputation());
4
5     if (ctx.query.isExactIn) {
6         uint256 takerDefinedAmountIn = ctx.swap.amountIn;
7         feeAmountIn = ctx.swap.amountIn * feeBps / BPS;
8         ctx.swap.amountIn -= feeAmountIn;
9         ctx.runLoop();
10        ctx.swap.amountIn = takerDefinedAmountIn;
11    } else {
12        ctx.runLoop();
13        feeAmountIn = ctx.swap.amountIn * feeBps / (BPS - feeBps); // @audit Division by zero if feeBps == BPS
14        ctx.swap.amountIn += feeAmountIn;
15    }
16 }
```

If `feeBps` is set equal to `BPS` (1e9, representing 100%), the denominator (`BPS - feeBps`) becomes zero, causing a division by zero panic.

While a 100% fee is not a realistic configuration for normal operation, the lack of input validation means a maker could accidentally configure this value.

Recommendation(s): Consider adding a validation to ensure `feeBps < BPS`.

Status: Acknowledged.

Update from the client: Takers must validate strategies before use.

6.13 [Info] Extruction targets can cause quote(...) and swap(...) divergence

File(s): swap-vm/src/instructions/Extruction.sol

Description: The Extruction instruction is designed to allow the SwapVM to execute custom logic by calling external contracts. This is facilitated through the `_extruction(...)` function, which makes an external call to a target address and allows that target to modify the current swap registers, program counter, and consume taker data.

The `_extruction(...)` implementation explicitly passes the `isStaticContext` flag as a parameter to the external target. The SwapVM defines this flag as true when the execution is triggered via the `quote(...)` function and false when triggered via the `swap(...)` function.

```

1  function _extruction(Context memory ctx, bytes calldata args) internal {
2      address target = address(bytes20(args.slice(0, 20, ExtructionMissingTargetArg.selector)));
3      uint256 choppedLength;
4
5      if (ctx.vm.isStaticContext) {
6          (ctx.vm.nextPC, choppedLength, ctx.swap) = IStaticExtruction(target).extruction(
7              // @audit Target receives the isStaticContext flag.
8              ctx.vm.isStaticContext,
9              ctx.vm.nextPC,
10             ctx.query,
11             ctx.swap,
12             args.slice(20),
13             ctx.takerArgs()
14         );
15     } else {
16         (ctx.vm.nextPC, choppedLength, ctx.swap) = IExtruction(target).extruction(
17             // @audit Target receives the isStaticContext flag.
18             ctx.vm.isStaticContext,
19             ctx.vm.nextPC,
20             ctx.query,
21             ctx.swap,
22             args.slice(20),
23             ctx.takerArgs()
24         );
25     }
26     // ...
27 }
28

```

Because the external target is aware of the execution context, it can deliberately return different values for a quote versus an actual swap.

Recommendation(s): Asses the need of having the Extruction instruction, if needed consider documenting the risks of this instruction around `quote(...)` and `swap(...)` results divergence.

Status: Acknowledged.

Update from the client: The Extruction extension is indeed potentially dangerous and cannot be made completely secure. In any case, a taker should validate a strategy that includes Extruction before using it.

6.14 [Info] Maker can manipulate swap rates via Self-Push in Aqua strategies without balance wrappers

File(s): `aqua/src/Aqua.sol`

Description: When Aqua-based strategies are created without balance wrapper instructions (`DynamicBalances`),

```

1 // AquaAMM.sol - buildProgram(...)
2 bytes memory bytecode = bytes.concat(
3   (delta0 != 0 || delta1 != 0) ? program.build(_xycConcentrateGrowLiquidity2D, ...) : bytes(""),
4   (decayPeriod > 0) ? program.build(_decayXD, ...) : bytes(""),
5   (feeBpsIn > 0) ? program.build(_flatFeeAmountInXD, ...) : bytes(""),
6   (protocolFeeBpsIn > 0) ? program.build(_aquaProtocolFeeAmountOutXD, ...) : bytes(""),
7   program.build(_xycSwapXD), // @audit Directly uses Aqua balances
8   // ...
9 );

```

the swap calculations directly use Aqua's virtual balances retrieved via `safeBalances(...)`.

```

1 // SwapVM.sol - swap(...)
2 if (order.traits.useAquaInsteadOfSignature()) {
3   (ctx.swap.balanceIn, ctx.swap.balanceOut) = AQUA.safeBalances(order.maker, address(this), orderHash, tokenIn,
4     ↳ tokenOut);
5 }
6 // @audit ... then ctx.runLoop() uses these balances for XYCSwap calculations

```

A maker can exploit this by calling `push(...)` on their own strategy with themselves as both `msg.sender` and maker, which inflates the virtual balance without any net token movement (self-transfer).

```

1 // Aqua.sol - push(...)
2 function push(address maker, address app, bytes32 strategyHash, address token, uint256 amount) external {
3   Balance storage balance = _balances[maker][app][strategyHash][token];
4   (uint248 prevBalance, uint8 tokensCount) = balance.load();
5   require(tokensCount > 0 && tokensCount != _DOCKED, PushToNonActiveStrategyPrevented(...));
6   balance.store(prevBalance + amount.toUint248(), tokensCount); // @audit Virtual balance increases
7
8   IERC20(token).safeTransferFrom(msg.sender, maker, amount); // @audit Self-transfer when msg.sender == maker
9   emit Pushed(maker, app, strategyHash, token, amount);
10 }

```

This allows the maker to front-run taker swaps and manipulate the exchange rate.

An example attack scenario is as follows:

1. Maker creates an Aqua strategy using the `AquaAMM.buildProgram(...)` or similar pattern that doesn't include balance wrapper instructions;
2. Maker ships the strategy with initial balances (e.g., 1000 tokenA, 1000 tokenB);
3. Taker queries `quote(...)` and receives an expected rate (e.g., 100 tokenA → 90 tokenB);
4. Taker submits swap transaction to the mempool;
5. Maker sees the pending transaction and front-runs with `push(maker, swapVM, strategyHash, tokenA, 9000)`;
6. The `push(...)` executes `safeTransferFrom(maker, maker, 9000)`;
7. Aqua's virtual balance for tokenA is now inflated to 10,000;
8. Taker's swap executes with the manipulated balances;
9. Taker receives 9 tokenB instead of the quoted 90 tokenB;
10. Maker can restore the balance by swapping with themselves in the opposite direction, push for the opposite token or they can dock the strategy;

It must be noted that takers can set a threshold in `TakerTraits` to specify minimum acceptable output. If set properly, the attack causes a revert rather than fund loss.

Recommendation(s): Consider documenting importance of using threshold for takers.

Status: Acknowledged.

Update from the client: The taker must validate strategies and set `minReturn` himself.

6.15 [Best Practices] decayPeriod can be set to zero which invalidates the decay mechanism

File(s): swap-vm/src/instructions/Decay.sol

Description: The Decay instruction provides MEV protection by adjusting virtual balances based on recent swap activity. The decayPeriod parameter controls how long offsets persist before fully decaying. However, there is no validation to prevent decayPeriod from being set to zero:

```
1 // Decay.sol
2 function getOffset(DecayingOffset storage self, uint256 decayPeriod) internal view returns (uint256) {
3     (uint216 offset, uint40 time) = _load(self);
4     uint256 expiration = time + decayPeriod;
5     if (block.timestamp >= expiration) {
6         return 0;
7     }
8     uint256 timeLeft = expiration - block.timestamp;
9     return offset * timeLeft / decayPeriod;
10 }
```

When decayPeriod = 0:

- expiration = time + 0 = time;
- Since time is set to block.timestamp when storing, block.timestamp >= time is always true;
- getOffset(...) always returns 0;
- The decay mechanism has no effect on swap calculations;

A maker who accidentally configures decayPeriod = 0 would believe their strategy has MEV protection when it does not. The strategy would function normally but without the intended protection.

Recommendation(s): Consider adding a validation to ensure a non-zero decay period.

Status: Acknowledged.

Update from the client: The maker must validate strategies before use.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- **Technical whitepaper:** A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- **User manual:** A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- **Code documentation:** Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- **API documentation:** API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- **Testing documentation:** Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- **Audit documentation:** Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Aqua and SwapVM documentation

The documentation for the contracts was provided through the code's Natspec and README files. Moreover, the protocol's development team addressed all questions and concerns raised by the Nethermind Security team during the kick-off call and through asynchronous communication, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Tests Output

8.1.1 aqua

```
> forge test

[] Compiling...
[] Compiling 45 files with Solc 0.8.30
[] Solc 0.8.30 finished in 2.49s
Compiler run successful!

Ran 8 tests for test/AquaStorageTest.t.sol:AquaStorageTest
[PASS] testDock1Token() (gas: 50947)
[PASS] testDock2Tokens() (gas: 80108)
[PASS] testDock3Tokens() (gas: 109363)
[PASS] testPullSingleSloadSstore() (gas: 67696)
[PASS] testPushSingleSloadSstore() (gas: 70616)
[PASS] testShip1Token() (gas: 46337)
[PASS] testShip2Tokens() (gas: 74317)
[PASS] testShip3Tokens() (gas: 102276)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 16.56ms (7.31ms CPU time)

Ran 18 tests for test/Aqua.t.sol:AquaTest
[PASS] testBalancesReturnsCorrectAmounts() (gas: 105159)
[PASS] testBalancesReturnsZeroForNonExistentStrategy() (gas: 14848)
[PASS] testBalancesReturnsZeroForTokenNotInStrategy() (gas: 50741)
[PASS] testDockRequiresAllTokensFromShip() (gas: 76825)
[PASS] testDockRequiresCorrectTokenCount() (gas: 79476)
[PASS] testDockRequiresExactTokensFromShip() (gas: 81174)
[PASS] testFullLifecycle() (gas: 144932)
[PASS] testMultipleStrategiesSameTokens() (gas: 169969)
[PASS] testPushFailsAfterDock() (gas: 54897)
[PASS] testPushOnlyForShippedTokens() (gas: 54277)
[PASS] testPushRequiresActiveStrategy() (gas: 21078)
[PASS] testSafeBalancesReturnsCorrectAmountsForActiveStrategy() (gas: 75692)
[PASS] testSafeBalancesRevertsAfterDock() (gas: 54832)
[PASS] testSafeBalancesRevertsForNonExistentStrategy() (gas: 20785)
[PASS] testSafeBalancesRevertsIfAnyTokenNotInStrategy() (gas: 79974)
[PASS] testSafeBalancesTracksChangesFromPushPull() (gas: 138495)
[PASS] testShipCannotBeCalledTwiceForSameStrategy() (gas: 52784)
[PASS] testShipCannotHaveDuplicateTokens() (gas: 46793)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 16.61ms (3.20ms CPU time)

Ran 2 test suites in 112.20ms (33.16ms CPU time): 26 tests passed, 0 failed, 0 skipped (26 total tests)
```

8.1.2 swap-vm

```
> forge test

[] Compiling...
[] Compiling 186 files with Solc 0.8.30
[] Solc 0.8.30 finished in 138.11s

Ran 6 tests for test/BaseFeeAdjuster.t.sol:BaseFeeAdjusterTest
[PASS] test_BaseFeeAdjusterDifferentEthPrices() (gas: 448366)
[PASS] test_BaseFeeAdjusterExactOut() (gas: 127992)
[PASS] test_BaseFeeAdjusterLimitSwapGasVariations() (gas: 317362)
[PASS] test_BaseFeeAdjusterMaxDecayLimits() (gas: 123995)
[PASS] test_BaseFeeAdjusterNoAdjustmentBelowBase() (gas: 158003)
[PASS] test_BaseFeeAdjusterWithDutchAuction() (gas: 555844)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 17.46ms (10.81ms CPU time)

Ran 8 tests for test/MinRate.t.sol:MinRateTest
[PASS] test_AdjustMinRateCapsOutput() (gas: 208535)
[PASS] test_AdjustMinRateExactOut() (gas: 122154)
[PASS] test_MinRateExtreme() (gas: 209195)
[PASS] test_MinRateNoEffectOnWorseRates() (gas: 208767)
[PASS] test_MinRateTokenOrdering() (gas: 374854)
```

```
[PASS] test_MinRateWithFees() (gas: 222143)
[PASS] test_RequireMinRatePass() (gas: 208115)
[PASS] test_RequireMinRateRevert() (gas: 159306)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 18.84ms (12.19ms CPU time)
```

```
Ran 3 tests for test/Decay.t.sol:DecayTest
[PASS] test_BasicDirections() (gas: 800609)
[PASS] test_DecayOverTime() (gas: 1209262)
[PASS] test_MEVSandwichProtection() (gas: 554247)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 19.38ms (12.72ms CPU time)
```

```
Ran 22 tests for test/gas/AMMGas.t.sol:AMMGas
[PASS] test_gas_ConcentrateGrowLiquidity_XYCSwap_quote_exactIn() (gas: 147209)
[PASS] test_gas_ConcentrateGrowLiquidity_XYCSwap_quote_exactOut() (gas: 147529)
[PASS] test_gas_ConcentrateGrowLiquidity_XYCSwap_swap_exactIn() (gas: 271981)
[PASS] test_gas_ConcentrateGrowLiquidity_XYCSwap_swap_exactOut() (gas: 273323)
[PASS] test_gas_ConcentrateGrowPriceRange_XYCSwap_quote_exactIn() (gas: 143921)
[PASS] test_gas_ConcentrateGrowPriceRange_XYCSwap_swap_exactIn() (gas: 244636)
[PASS] test_gas_ConcentrateDecay_XYCSwap_quote_exactIn() (gas: 163721)
[PASS] test_gas_ConcentrateDecay_XYCSwap_swap_exactIn() (gas: 336965)
[PASS] test_gas_Decay_XYCSwap_quote_exactIn() (gas: 131291)
[PASS] test_gas_Decay_XYCSwap_quote_exactOut() (gas: 132227)
[PASS] test_gas_Decay_XYCSwap_swap_exactIn() (gas: 279763)
[PASS] test_gas_Decay_XYCSwap_swap_exactOut() (gas: 279950)
[PASS] test_gas_FullAMM_quote_exactIn() (gas: 177086)
[PASS] test_gas_FullAMM_swap_exactIn() (gas: 349758)
[PASS] test_gas_XYCSwap_FlatFeeIn_quote_exactIn() (gas: 127493)
[PASS] test_gas_XYCSwap_FlatFeeIn_swap_exactIn() (gas: 229396)
[PASS] test_gas_XYCSwap_FlatFeeOut_quote_exactIn() (gas: 127827)
[PASS] test_gas_XYCSwap_FlatFeeOut_swap_exactIn() (gas: 228715)
[PASS] test_gas_XYCSwap_quote_exactIn() (gas: 114010)
[PASS] test_gas_XYCSwap_quote_exactOut() (gas: 115373)
[PASS] test_gas_XYCSwap_swap_exactIn() (gas: 216034)
[PASS] test_gas_XYCSwap_swap_exactOut() (gas: 216109)
Suite result: ok. 22 passed; 0 failed; 0 skipped; finished in 22.56ms (15.72ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

```
Ran 4 tests for test/DutchAuction.t.sol:DutchAuctionTest
[PASS] test_DutchAuctionIn_DecayFactors() (gas: 1785282)
[PASS] test_DutchAuctionIn_Expiry() (gas: 157658)
[PASS] test_DutchAuctionOut_DecayFactors() (gas: 1787092)
[PASS] test_DutchAuctionOut_Expiry() (gas: 157675)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 24.94ms (18.30ms CPU time)
```

```
Ran 23 tests for test/TakerTraits.t.sol:TakerTraitsTest
[PASS] test_AllDataSlices_Populated() (gas: 221277)
[PASS] test_Build_EmptyThreshold_Valid() (gas: 13434)
[PASS] test_Build_ValidThreshold_32Bytes() (gas: 13894)
[PASS] test_CombinedFeatures_DeadlineAndThreshold() (gas: 221580)
[PASS] test_Deadline_AtCurrentTimestamp_Success() (gas: 213673)
[PASS] test_Deadline_Expired_Reverts() (gas: 124953)
[PASS] test_Deadline_NotSet_Success() (gas: 214034)
[PASS] test_Deadline_Valid_Success() (gas: 214843)
[PASS] test_ExactIn_MinThreshold_Fails() (gas: 128420)
[PASS] test_ExactIn_MinThreshold_Success() (gas: 215230)
[PASS] test_ExactIn_TakerAmountMismatch() (gas: 214217)
[PASS] test_ExactOut_MaxThreshold_Fails() (gas: 128597)
[PASS] test_ExactOut_MaxThreshold_Success() (gas: 215303)
[PASS] test_ExactOut_TakerAmountMatch() (gas: 213929)
[PASS] test_IsFirstTransferFromTaker_False() (gas: 214500)
[PASS] test_IsFirstTransferFromTaker_True() (gas: 223068)
[PASS] test_StrictThreshold_ExactMatch_Success() (gas: 214673)
[PASS] test_StrictThreshold_Mismatch_Fails() (gas: 128773)
[PASS] test_TakerHookData_EmptyWithMakerHooks() (gas: 1593746)
[PASS] test_TakerHookData_OnlyPreTransferIn() (gas: 1393338)
[PASS] test_TakerHookData_PassedToHooks() (gas: 2140803)
[PASS] test_To_CustomRecipient() (gas: 227459)
[PASS] test_To_NotSet_SendsToTaker() (gas: 224843)
Suite result: ok. 23 passed; 0 failed; 0 skipped; finished in 29.86ms (23.20ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
revertTo(uint256): replaced by `revertToState`
snapshot(): replaced by `snapshotState`
```

```

Ran 9 tests for test/invariants/BaseFeeAdjusterFeesInvariants.t.sol:BaseFeeAdjusterFeesInvariants
[PASS] test_BaseFeeAdjuster_DutchAuctionIn_FlatFeeIn() (gas: 1996626)
[PASS] test_BaseFeeAdjuster_DutchAuctionIn_ProtocolFee() (gas: 2188557)
[PASS] test_BaseFeeAdjuster_DutchAuctionOut_FlatFeeOut() (gas: 2001237)
[PASS] test_BaseFeeAdjuster_DutchAuctionOut_MultipleFees() (gas: 2284561)
[PASS] test_BaseFeeAdjuster_FlatFeeIn() (gas: 1790550)
[PASS] test_BaseFeeAdjuster_FlatFeeOut() (gas: 1793159)
[PASS] test_BaseFeeAdjuster_HighFees() (gas: 1790440)
[PASS] test_BaseFeeAdjuster_MultipleFees() (gas: 2075957)
[PASS] test_BaseFeeAdjuster_ProtocolFee() (gas: 1983144)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 75.20ms (68.54ms CPU time)

Ran 5 tests for test/ProtocolFeeAqua.t.sol:ProtocolFeeAquaTest
[PASS] test_Aqua_ProtocolFee_ExactIn_ReceivedByRecipient() (gas: 495755)
[PASS] test_Aqua_ProtocolFee_ExactIn_WithFlatFeeIn() (gas: 506377)
[PASS] test_Aqua_ProtocolFee_ExactOut_ReceivedByRecipient() (gas: 517878)
[PASS] test_Aqua_ProtocolFee_ExactOut_WithFlatFeeIn() (gas: 528039)
[PASS] test_Aqua_ProtocolFee_WithFlatFeeInAndOut_Consistency() (gas: 439155)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 4.12ms (3.58ms CPU time)

Ran 3 tests for test/ProtocolFeeProviderMock.t.sol:ProtocolFeeProviderMockTest
[PASS] testGetProtocolFeeParams() (gas: 18257)
[PASS] testSetProtocolFeeParams() (gas: 23214)
[PASS] testSetProtocolFeeParams_NotOwner() (gas: 12984)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 192.25µs (86.46µs CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 9 tests for test/invariants/pegged/BalancedCurve.t.sol:BalancedCurve
[PASS] test_Pegged() (gas: 6130255)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7403832)
[PASS] test_PeggedFlatFeeIn() (gas: 6274260)
[PASS] test_PeggedFlatFeeOut() (gas: 3045638)
[PASS] test_PeggedMultipleFees() (gas: 3043747)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2670367)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2671010)
[PASS] test_PeggedProtocolFee() (gas: 6786932)
[PASS] test_PeggedProtocolFeeIn() (gas: 6897546)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 85.88ms (132.22ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/pegged/BalancedPoolEdgeFees.t.sol:BalancedPoolEdgeFees
[PASS] test_Pegged() (gas: 5594628)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 6801424)
[PASS] test_PeggedFlatFeeIn() (gas: 5722308)
[PASS] test_PeggedFlatFeeOut() (gas: 2475888)
[PASS] test_PeggedMultipleFees() (gas: 2419438)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2097078)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2097631)
[PASS] test_PeggedProtocolFee() (gas: 6213623)
[PASS] test_PeggedProtocolFeeIn() (gas: 6323622)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 90.73ms (84.08ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 7 tests for test/invariants/DecayXYCFeesInvariants.t.sol:DecayXYCFeesInvariants
[PASS] test_DecayXYCFlatFeeIn() (gas: 3473087)
[PASS] test_DecayXYCFlatFeeOut() (gas: 1940101)
[PASS] test_DecayXYCMultipleFees() (gas: 2858819)
[PASS] test_DecayXYCProgressiveFeeIn() (gas: 2739658)
[PASS] test_DecayXYCProgressiveFeeOut() (gas: 2031724)
[PASS] test_DecayXYCProtocolFee() (gas: 2038057)
[PASS] test_DecayXYCProtocolFeeIn() (gas: 3699268)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 75.86ms (74.68ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 7 tests for test/invariants/MinRateInvariants.t.sol:MinRateInvariants
[PASS] test_MinRate_DutchAuctionIn_LimitSwap() (gas: 4614898)
[PASS] test_MinRate_DutchAuctionOut_LimitSwap() (gas: 4440380)

```

```
[PASS] test_MinRate_DutchAuction_LimitSwap_Fees() (gas: 3844554)
[PASS] test_MinRate_LimitSwap() (gas: 1873164)
[PASS] test_MinRate_LimitSwap_FlatFeeIn() (gas: 3218349)
[PASS] test_MinRate_LimitSwap_FlatFeeOut() (gas: 3219075)
[PASS] test_MinRate_LimitSwap_ProtocolFee() (gas: 3436874)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 77.38ms (76.76ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 snapshot(): replaced by `snapshotState`
 revertTo(uint256): replaced by `revertToState`

```
Ran 5 tests for test/invariants/DecayXYCInvariants.t.sol:DecayXYCInvariants
[PASS] test_DecayXYCDifferentPeriods() (gas: 12303968)
[PASS] test_DecayXYCHalfwayDecay() (gas: 3101392)
[PASS] test_DecayXYCLargePeriod() (gas: 3101482)
[PASS] test_DecayXYCMediumPeriod() (gas: 3101284)
[PASS] test_DecayXYCSmallPeriod() (gas: 3101108)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 80.97ms (80.39ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 snapshot(): replaced by `snapshotState`
 revertTo(uint256): replaced by `revertToState`

```
Ran 6 tests for test/invariants/BaseFeeAdjusterInvariants.t.sol:BaseFeeAdjusterInvariants
[PASS] test_BaseFeeAdjuster_DifferentEthPrices() (gas: 5030274)
[PASS] test_BaseFeeAdjuster_HighGas() (gas: 1697524)
[PASS] test_BaseFeeAdjuster_LowGas() (gas: 4137230)
[PASS] test_BaseFeeAdjuster_ModerateGas() (gas: 1697480)
[PASS] test_BaseFeeAdjuster_WithDutchAuctionIn() (gas: 1903385)
[PASS] test_BaseFeeAdjuster_WithDutchAuctionOut() (gas: 1905453)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 38.82ms (48.37ms CPU time)
```

```
Ran 2 tests for test/SwapVM.t.sol:SwapVMTest
[PASS] test_LimitSwapWithTokenOutInvalidator() (gas: 598592)
[PASS] test_LimitSwapWithoutInvalidator_ReusableOrder() (gas: 265511)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.14ms (1.59ms CPU time)
```

```
Ran 2 tests for test/SwapVMAqua.t.sol:SwapVMAquaTest
[PASS] test_Aqua_XYC_SimpleSwap() (gas: 416708)
[PASS] test_Aqua_XYC_SwapWithFirstTransferFromTaker() (gas: 1026082)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.24ms (751.71µs CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 revertTo(uint256): replaced by `revertToState`
 snapshot(): replaced by `snapshotState`

```
Ran 9 tests for test/invariants/pegged/TinyLiquidity.t.sol:TinyLiquidity
[PASS] test_Pegged() (gas: 6149901)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7423479)
[PASS] test_PeggedFlatFeeIn() (gas: 6293906)
[PASS] test_PeggedFlatFeeOut() (gas: 3061898)
[PASS] test_PeggedMultipleFees() (gas: 3058915)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2675386)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2675417)
[PASS] test_PeggedProtocolFee() (gas: 6806608)
[PASS] test_PeggedProtocolFeeIn() (gas: 6917192)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 124.52ms (123.90ms CPU time)
```

```
Ran 7 tests for test/TWAPSwap.t.sol:TWAPSwapTest
[PASS] test_TWAPBoundaryConditions() (gas: 322010)
[PASS] test_TWAPExponentialDecay() (gas: 368354)
[PASS] test_TWAPIlliquidityBumpCalculation() (gas: 369277)
[PASS] test_TWAPLinearUnlocking() (gas: 365846)
[PASS] test_TWAPMinimumTradeAmount() (gas: 357484)
[PASS] test_TWAPPriceBumpAfterIlliquidity() (gas: 286025)
[PASS] test_TWAPSequentialTrades() (gas: 452079)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 11.26ms (10.72ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 snapshot(): replaced by `snapshotState`
 revertTo(uint256): replaced by `revertToState`

```
Ran 9 tests for test/invariants/pegged/PureSquareRoot.t.sol:PureSquareRoot
[PASS] test_Pegged() (gas: 5862367)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7135944)
[PASS] test_PeggedFlatFeeIn() (gas: 6006372)
[PASS] test_PeggedFlatFeeOut() (gas: 2878208)
[PASS] test_PeggedMultipleFees() (gas: 2909803)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2536423)
```

```
[PASS] test_PeggedProgressiveFeeOut() (gas: 2537066)
[PASS] test_PeggedProtocolFee() (gas: 6519102)
[PASS] test_PeggedProtocolFeeIn() (gas: 6629658)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 108.32ms (107.72ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 7 tests for test/invariants/TWAPLimitSwapInvariants.t.sol:TWAPLimitSwapInvariants
[PASS] test_TWAP_BasicInvariants() (gas: 4847706)
[PASS] test_TWAP_FlatFeeIn() (gas: 4796282)
[PASS] test_TWAP_FlatFeeOut() (gas: 2086656)
[PASS] test_TWAP_HighPriceBumpWithFees() (gas: 1382451)
[PASS] test_TWAP_MultipleFees() (gas: 2666375)
[PASS] test_TWAP_ProtocolFee() (gas: 1645510)
[PASS] test_TWAP_TimeProgressionWithFees() (gas: 6947346)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 61.79ms (61.22ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 9 tests for test/invariants/pegged/DustAmounts.t.sol:DustAmounts
[PASS] test_Pegged() (gas: 5398968)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 6583507)
[PASS] test_PeggedFlatFeeIn() (gas: 5501614)
[PASS] test_PeggedFlatFeeOut() (gas: 2269502)
[PASS] test_PeggedMultipleFees() (gas: 2174979)
[PASS] test_PeggedProgressiveFeeIn() (gas: 1890529)
[PASS] test_PeggedProgressiveFeeOut() (gas: 1890860)
[PASS] test_PeggedProtocolFee() (gas: 6005568)
[PASS] test_PeggedProtocolFeeIn() (gas: 6115200)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 105.82ms (83.02ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 14 tests for test/invariants/ConcentrateXYCDelayFeesInvariants.t.sol:ConcentrateXYCDelayFeesInvariants
[PASS] test_Order1_GrowLiquidity2D() (gas: 3047994)
[PASS] test_Order1_GrowLiquidityXD() (gas: 3317586)
[PASS] test_Order1_GrowPriceRange2D() (gas: 2560753)
[PASS] test_Order1_GrowPriceRangeXD() (gas: 2715777)
[PASS] test_Order2_GrowLiquidity2D() (gas: 3048008)
[PASS] test_Order2_GrowPriceRange2D() (gas: 2559709)
[PASS] test_Order3_GrowLiquidityXD() (gas: 3121941)
[PASS] test_Order3_GrowPriceRangeXD() (gas: 2929982)
[PASS] test_Order4_GrowLiquidity2D() (gas: 3243393)
[PASS] test_Order4_GrowPriceRange2D() (gas: 2656577)
[PASS] test_Order5_GrowLiquidityXD() (gas: 3121640)
[PASS] test_Order5_GrowPriceRangeXD() (gas: 2929584)
[PASS] test_Order6_GrowLiquidity2D() (gas: 3049182)
[PASS] test_Order6_GrowPriceRange2D() (gas: 3168425)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 120.45ms (119.58ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 17 tests for test/PeggedSwap.t.sol:PeggedSwapTest
[PASS] test_EdgeCase_99Percent_Unbalanced_Pool() (gas: 1402810)
[PASS] test_EdgeCase_A_Max_MinimalSlippage() (gas: 2262861)
[PASS] test_EdgeCase_A_Zero_PureSquareRoot() (gas: 2263874)
[PASS] test_LargeA_PeggedAssets_LargeSwap() (gas: 766028)
[PASS] test_LargeA_PeggedAssets_MediumSwap() (gas: 746671)
[PASS] test_LargeA_PeggedAssets_SmallSwap() (gas: 689485)
[PASS] test_PeggedSwap_ActualOnchainSwap_ExactIn() (gas: 953179)
[PASS] test_PeggedSwap_ImbalancedPool_NoFee_ShowsSlippage() (gas: 918554)
[PASS] test_PeggedSwap_NoFee_vs_WithFee_Comparison() (gas: 365753)
[PASS] test_PeggedSwap_RoundingInvariantsWithFees() (gas: 13900093)
[PASS] test_PeggedSwap_WithFee_0_3_Percent_ExactIn() (gas: 213410)
[PASS] test_RoundingEffectOnLargeSwaps() (gas: 93404)
[PASS] test_RoundingProtectsProtocol_ExactIn() (gas: 35339)
[PASS] test_RoundingProtectsProtocol_ExactOut() (gas: 34556)
[PASS] test_SmallA_UnpeggedAssets_LargeSwap() (gas: 766057)
[PASS] test_SmallA_UnpeggedAssets_MediumSwap() (gas: 746921)
[PASS] test_SmallA_UnpeggedAssets_SmallSwap() (gas: 689809)
Suite result: ok. 17 passed; 0 failed; 0 skipped; finished in 108.14ms (107.49ms CPU time)
```

```

Ran 14 tests for test/ProgressiveFee.t.sol:ProgressiveFeeTest
[PASS] test_ProgressiveFeeIn_ConsistentForExactInAndExactOut() (gas: 185903)
[PASS] test_ProgressiveFeeIn_ExactIn_DecreasesBySplittingAmount() (gas: 306357)
[PASS] test_ProgressiveFeeIn_ExactIn_IncreasesWithLargerSwaps() (gas: 170394)
[PASS] test_ProgressiveFeeIn_ExactIn_ProvidesMoreFairRateThanFlatFees() (gas: 3358296)
[PASS] test_ProgressiveFeeIn_ExactIn_WithFlatFees() (gas: 349688)
[PASS] test_ProgressiveFeeIn_ExactIn_ZeroFeeBehavesLikeNoFeeStrategy() (gas: 216153)
[PASS] test_ProgressiveFeeIn_ExactOut_DecreasesBySplittingAmount() (gas: 308644)
[PASS] test_ProgressiveFeeIn_ExactOut_IncreasesWithLargerSwaps() (gas: 171167)
[PASS] test_ProgressiveFeeIn_ExactOut_ProvidesMoreFairRateThanFlatFees() (gas: 1621023)
[PASS] test_ProgressiveFeeIn_ExactOut_WithFlatFees() (gas: 352166)
[PASS] test_ProgressiveFeeIn_ExactOut_ZeroFeeBehavesLikeNoFeeStrategy() (gas: 218536)
[PASS] test_ProgressiveFeeIn_InvariantGrowthScalesWithPriceImpact() (gas: 345977)
[PASS] test_ProgressiveFee_MaintainsSymmetry() (gas: 418223)
[PASS] test_SymmetryInvariant_ZeroProgressiveFee() (gas: 417614)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 46.05ms (43.67ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 7 tests for test/invariants/DutchAuctionLimitSwapFeesInvariants.t.sol:DutchAuctionLimitSwapFeesInvariants
[PASS] test_DutchAuctionIn_FlatFeeIn() (gas: 13174032)
[PASS] test_DutchAuctionIn_HighFees() (gas: 13173944)
[PASS] test_DutchAuctionIn_ProgressiveFeeIn() (gas: 5780547)
[PASS] test_DutchAuctionIn_ProtocolFee() (gas: 6792209)
[PASS] test_DutchAuctionOut_FlatFeeOut() (gas: 13185417)
[PASS] test_DutchAuctionOut_MultipleFees() (gas: 6575974)
[PASS] test_DutchAuctionOut_ProgressiveFeeOut() (gas: 6245974)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 190.96ms (189.68ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 5 tests for test/invariants/ConcentrateXYCDelayInvariants.t.sol:ConcentrateXYCDelayInvariants
[PASS] test_ConcentrateGrowLiquidity2D_Decay() (gas: 6739382)
[PASS] test_ConcentrateGrowLiquidityXD_Decay() (gas: 6847208)
[PASS] test_ConcentrateGrowPriceRange2D_Decay() (gas: 6205452)
[PASS] test_ConcentrateGrowPriceRangeXD_Decay() (gas: 6305452)
[PASS] test_ConcentrateVariousDecayPeriods() (gas: 26816853)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 120.82ms (119.68ms CPU time)

Ran 9 tests for test/DynamicProtocolFee.t.sol:DynamicProtocolFeeTest
[PASS] test_DynamicProtocolFee_ExactIn_ReceivedByRecipient() (gas: 299991)
[PASS] test_DynamicProtocolFee_ExactOut_ReceivedByRecipient() (gas: 302923)
[PASS] test_DynamicProtocolFee_ProviderCanChangeFee() (gas: 414196)
[PASS] test_DynamicProtocolFee_ProviderReturnsFailedCall_Reverts() (gas: 138475)
[PASS] test_DynamicProtocolFee_ProviderReturnsHighFee_Reverts() (gas: 150573)
[PASS] test_DynamicProtocolFee_WithFlatFee() (gas: 312301)
[PASS] test_DynamicProtocolFee_ZeroAddress_Reverts() (gas: 148698)
[PASS] test_DynamicProtocolFee_ZeroFee_NoTransfer() (gas: 272134)
[PASS] test_DynamicProtocolFee_ZeroProvider_NoFee() (gas: 255336)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 5.43ms (4.74ms CPU time)

Ran 7 tests for test/ProtocolFee.t.sol:ProtocolFeeTest
[PASS] test_ProtocolFeeAmountIn_ExactIn_ReceivedByRecipient() (gas: 286362)
[PASS] test_ProtocolFeeAmountIn_ExactOut_ReceivedByRecipient() (gas: 289634)
[PASS] test_ProtocolFeeAmountIn_WithFlatFeeIn_ExactIn() (gas: 298050)
[PASS] test_ProtocolFee_ExactIn_WithFlatFeeGivesWorseRate() (gas: 392898)
[PASS] test_ProtocolFee_ExactOut_WithFlatFeeGivesWorseRate() (gas: 393965)
[PASS] test_ProtocolFee_Only_ExactIn_ReceivedByRecipient() (gas: 283060)
[PASS] test_ProtocolFee_Only_ExactOut_ReceivedByRecipient() (gas: 283953)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 10.24ms (6.88ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 18 tests for test/invariants/ConcentrateXYCFeesInvariants.t.sol:ConcentrateXYCFeesInvariants
[PASS] test_ConcentrateXYCDynamicProtocolFeeIn() (gas: 6912946)
[PASS] test_ConcentrateXYCFlatFeeIn() (gas: 5765063)
[PASS] test_ConcentrateXYCFlatFeeIn_GrowLiquidityXD() (gas: 5877814)
[PASS] test_ConcentrateXYCFlatFeeIn_GrowPriceRange() (gas: 5220855)
[PASS] test_ConcentrateXYCFlatFeeOut() (gas: 2610071)
[PASS] test_ConcentrateXYCFlatFeeOut_GrowPriceRange() (gas: 2411243)
[PASS] test_ConcentrateXYCFlatFeeOut_GrowPriceRangeXD() (gas: 2482374)

```

```
[PASS] test_ConcentrateXYCMultipleFees() (gas: 3027303)
[PASS] test_ConcentrateXYCMultipleFees_GrowPriceRange() (gas: 2830165)
[PASS] test_ConcentrateXYCProgressiveFeeIn() (gas: 2330359)
[PASS] test_ConcentrateXYCProgressiveFeeIn_GrowPriceRange() (gas: 2144751)
[PASS] test_ConcentrateXYCProgressiveFeeOut() (gas: 2331415)
[PASS] test_ConcentrateXYCProgressiveFeeOut_GrowLiquidityXD() (gas: 2390081)
[PASS] test_ConcentrateXYCProgressiveFeeOut_GrowPriceRange() (gas: 2145290)
[PASS] test_ConcentrateXYCProtocolFee() (gas: 6296478)
[PASS] test_ConcentrateXYCProtocolFeeIn() (gas: 6407071)
[PASS] test_ConcentrateXYCProtocolFee_GrowPriceRange() (gas: 5746512)
[PASS] test_ConcentrateXYCProtocolFee_GrowPriceRangeXD() (gas: 5926645)
Suite result: ok. 18 passed; 0 failed; 0 skipped; finished in 228.46ms (221.78ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 snapshot(): replaced by `snapshotState`
 revertTo(uint256): replaced by `revertToState`

Ran 9 tests for test/invariants/pegged/MostlyCurved.t.sol:MostlyCurved

```
[PASS] test_Pegged() (gas: 6140878)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7414456)
[PASS] test_PeggedFlatFeeIn() (gas: 6284883)
[PASS] test_PeggedFlatFeeOut() (gas: 3056231)
[PASS] test_PeggedMultipleFees() (gas: 3054332)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2670367)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2671010)
[PASS] test_PeggedProtocolFee() (gas: 6797614)
[PASS] test_PeggedProtocolFeeIn() (gas: 6908169)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 136.18ms (135.57ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 revertTo(uint256): replaced by `revertToState`
 snapshot(): replaced by `snapshotState`

Ran 4 tests for test/invariants/PeggedSwapInvariants.t.sol:PeggedSwapInvariants

```
[PASS] test_PeggedSwap_Invariants() (gas: 5999103)
[PASS] test_PeggedSwap_LargeOddAmounts() (gas: 700917)
[PASS] test_PeggedSwap_OddAmountRounding() (gas: 818178)
[PASS] test_PeggedSwap_RoundingConsistency() (gas: 1233027)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 35.93ms (35.38ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 revertTo(uint256): replaced by `revertToState`
 snapshot(): replaced by `snapshotState`

Ran 8 tests for test/Controls.t.sol:ControlsTest

```
[PASS] test_Deadline() (gas: 276498)
[PASS] test_Jump() (gas: 206753)
[PASS] test_JumpIfTokenIn() (gas: 367560)
[PASS] test_JumpIfTokenOut() (gas: 367881)
[PASS] test_OnlyTakerTokenBalanceGte() (gas: 325985)
[PASS] test_OnlyTakerTokenBalanceNonZero() (gas: 323524)
[PASS] test_OnlyTakerTokenSupplyShareGte() (gas: 330727)
[PASS] test_Salt() (gas: 353977)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 9.10ms (8.51ms CPU time)
```

Ran 4 tests for test/ControlsAqua.t.sol:ControlsAquaTest

```
[PASS] test_DeadlineAlreadyPassed() (gas: 245644)
[PASS] test_DeadlineControl() (gas: 404438)
[PASS] test_OnlyTakerTokenBalanceNonZero_Fail() (gas: 268330)
[PASS] test_OnlyTakerTokenBalanceNonZero_Success() (gas: 440929)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.08ms (3.10ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
 snapshot(): replaced by `snapshotState`
 revertTo(uint256): replaced by `revertToState`

Ran 5 tests for test/invariants/ExampleInvariantUsage.t.sol:ExampleInvariantUsage

```
[PASS] test_AMMWithFeesInvariants() (gas: 6524720)
[PASS] test_LimitOrderInvariants() (gas: 3946404)
[PASS] test_ProgressiveFeeInvariants() (gas: 2275285)
[PASS] test_SkipCertainInvariants() (gas: 1700833)
[PASS] test_SpecificInvariants() (gas: 310823)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 32.63ms (32.08ms CPU time)
```

Ran 12 tests for test/Fee.t.sol:FeeTest

```
[PASS] test_FeeIn_ExactIn_SwapAndQuoteConsistent() (gas: 271752)
[PASS] test_FeeIn_ExactOut_SwapAndQuoteConsistent() (gas: 272724)
[PASS] test_FeeIn_ExchangeRateConsistency() (gas: 183876)
[PASS] test_FeeIn_ExchangeRateConsistency_ForLargeAmount() (gas: 222506)
```

```
[PASS] test_FeeIn_FeeOut_ConsistentExactInVsExactOut() (gas: 200751)
[PASS] test_FeeOut_ExactIn_SwapAndQuoteConsistent() (gas: 271728)
[PASS] test_FeeOut_ExactOut_SwapAndQuoteConsistent() (gas: 273201)
[PASS] test_FeeOut_ExchangeRateConsistency() (gas: 184273)
[PASS] test_FeeOut_ExchangeRateConsistency_ForLargeAmount() (gas: 223196)
[PASS] test_SymmetryInvariant_BothFees() (gas: 368898)
[PASS] test_SymmetryInvariant_FeeIn() (gas: 338187)
[PASS] test_SymmetryInvariant_FeeOut() (gas: 338737)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 6.99ms (6.35ms CPU time)
```

```
Ran 10 tests for test/FeeAqua.t.sol:FeeAquaTest
[PASS] test_Aqua_FeeIn_ExactIn_100Percent_ShouldRevert() (gas: 343593)
[PASS] test_Aqua_FeeIn_ExactIn_BalanceAfterSwap() (gas: 446922)
[PASS] test_Aqua_FeeIn_ExactOut_100Percent_ShouldRevert() (gas: 341851)
[PASS] test_Aqua_FeeIn_ExactOut_BalancesAfterSwap() (gas: 469429)
[PASS] test_Aqua_FeeIn_FeeOut_ExactIn_AppliedSimultaneously_BalancesAfterSwap() (gas: 457712)
[PASS] test_Aqua_FeeIn_FeeOut_ExactOut_AppliedSimultaneously_BalancesAfterSwap() (gas: 478690)
[PASS] test_Aqua_FeeOut_ExactIn_100Percent_ShouldRevert() (gas: 343120)
[PASS] test_Aqua_FeeOut_ExactIn_BalancesAfterSwap() (gas: 448046)
[PASS] test_Aqua_FeeOut_ExactOut_100Percent_InfiniteAmountIn() (gas: 228695)
[PASS] test_Aqua_FeeOut_ExactOut_BalancesAfterSwap() (gas: 469599)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 6.01ms (5.52ms CPU time)
```

```
Ran 14 tests for test/FeeIndifferencyToSwap.t.sol:FeeIndifferencyToSwap
[PASS] test_FeeIn_WithCircularFormula() (gas: 59328)
[PASS] test_FeeIn_WithConstantSumFormula() (gas: 51085)
[PASS] test_FeeIn_WithHyperbolicFormula() (gas: 52853)
[PASS] test_FeeIn_WithInverseFormula() (gas: 53988)
[PASS] test_FeeIn_WithLinearFormula() (gas: 52350)
[PASS] test_FeeIn_WithSmoothTransitionFormula() (gas: 53605)
[PASS] test_FeeIn_WithXYCFormula() (gas: 51709)
[PASS] test_FeeOut_WithCircularFormula() (gas: 59166)
[PASS] test_FeeOut_WithConstantSumFormula() (gas: 50890)
[PASS] test_FeeOut_WithHyperbolicFormula() (gas: 52438)
[PASS] test_FeeOut_WithInverseFormula() (gas: 54112)
[PASS] test_FeeOut_WithLinearFormula() (gas: 52408)
[PASS] test_FeeOut_WithSmoothTransitionFormula() (gas: 53717)
[PASS] test_FeeOut_WithXYCFormula() (gas: 52174)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 2.28ms (2.19ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

```
Ran 4 tests for test/FeeOutAdditivityViolation.t.sol:FeeOutAdditivityViolation
[PASS] test_FeeIn_vs_FeeOut_Additivity() (gas: 609915)
[PASS] test_FeeOut_MinProfitableSwapSize() (gas: 10339404)
[PASS] test_FeeOut_SplitProfitability() (gas: 374415)
[PASS] test_FlatFeeOut_Original_ViolatesAdditivity() (gas: 355303)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 36.38ms (35.19ms CPU time)
```

```
Ran 8 tests for test/Invalidators.t.sol:InvalidatorsTest
[PASS] test_CombinedInvalidators() (gas: 342669)
[PASS] test_ExternalInvalidation() (gas: 308271)
[PASS] test_InvalidateBitDifferentIndices() (gas: 364499)
[PASS] test_InvalidateBitSingleUse() (gas: 279704)
[PASS] test_InvalidateBitSlotBoundaries() (gas: 835295)
[PASS] test_InvalidateTokenInPartialFills() (gas: 429443)
[PASS] test_InvalidateTokenOutPartialFills() (gas: 578906)
[PASS] test_InvalidatorZeroAmount() (gas: 151692)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 7.09ms (6.05ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

```
Ran 9 tests for test/invariants/PeggedFeesInvariants.t.sol:PeggedFeesInvariants
[PASS] test_Pegged() (gas: 6150958)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7424360)
[PASS] test_PeggedFlatFeeIn() (gas: 6294963)
[PASS] test_PeggedFlatFeeOut() (gas: 3062531)
[PASS] test_PeggedMultipleFees() (gas: 3059372)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2675407)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2676050)
[PASS] test_PeggedProtocolFee() (gas: 6807459)
[PASS] test_PeggedProtocolFeeIn() (gas: 6918073)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 85.24ms (84.21ms CPU time)
```

```
Ran 6 tests for test/UnwrapWeth.t.sol:UnwrapWethTest
[PASS] test_MakerShouldRecvEth() (gas: 285442)
[PASS] test_MakerShouldRecvEthToCustomAddress() (gas: 286974)
[PASS] test_RejectDirectEtherTransfer() (gas: 19356)
[PASS] test_TakerShouldRecvEth() (gas: 325377)
[PASS] test_TakerShouldRecvEthToCustomAddress() (gas: 326797)
[PASS] test_UnwrapWeth_Fuzz(bool,bool,bool,uint128) (runs: 256, : 315027, ~: 322593)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 172.24ms (171.63ms CPU time)
```

```
Ran 36 tests for test/gas/LimitSwapGas.t.sol:LimitSwapGas
[PASS] test_gas_Deadline_LimitSwap_quote_exactIn() (gas: 116456)
[PASS] test_gas_Deadline_LimitSwap_swap_exactIn() (gas: 174678)
[PASS] test_gas_DutchAuctionIn_LimitSwap_quote_exactIn() (gas: 122664)
[PASS] test_gas_DutchAuctionIn_LimitSwap_quote_exactOut() (gas: 123577)
[PASS] test_gas_DutchAuctionIn_LimitSwap_swap_exactIn() (gas: 181439)
[PASS] test_gas_DutchAuctionIn_LimitSwap_swap_exactOut() (gas: 182066)
[PASS] test_gas_DutchAuctionOut_LimitSwap_quote_exactIn() (gas: 123137)
[PASS] test_gas_DutchAuctionOut_LimitSwap_quote_exactOut() (gas: 122632)
[PASS] test_gas_DutchAuctionOut_LimitSwap_swap_exactIn() (gas: 181230)
[PASS] test_gas_DutchAuctionOut_LimitSwap_swap_exactOut() (gas: 181638)
[PASS] test_gas_FullLimitSwap_quote_exactIn() (gas: 150083)
[PASS] test_gas_FullLimitSwap_swap_exactIn() (gas: 230010)
[PASS] test_gas_InvalidateBit_LimitSwap_quote_exactIn() (gas: 120820)
[PASS] test_gas_InvalidateBit_LimitSwap_swap_exactIn() (gas: 199881)
[PASS] test_gas_LimitSwap_FlatFeeIn_quote_exactIn() (gas: 122795)
[PASS] test_gas_LimitSwap_FlatFeeIn_swap_exactIn() (gas: 180458)
[PASS] test_gas_LimitSwap_FlatFeeOut_quote_exactIn() (gas: 123335)
[PASS] test_gas_LimitSwap_FlatFeeOut_swap_exactIn() (gas: 180878)
[PASS] test_gas_LimitSwap_InvalidateTokenIn_quote_exactIn() (gas: 120772)
[PASS] test_gas_LimitSwap_InvalidateTokenIn_swap_exactIn() (gas: 199555)
[PASS] test_gas_LimitSwap_ProgressiveFee_quote_exactIn() (gas: 122548)
[PASS] test_gas_LimitSwap_ProgressiveFee_swap_exactIn() (gas: 181609)
[PASS] test_gas_LimitSwap_quote_exactIn() (gas: 108936)
[PASS] test_gas_LimitSwap_quote_exactOut() (gas: 110706)
[PASS] test_gas_LimitSwap_swap_exactIn() (gas: 168415)
[PASS] test_gas_LimitSwap_swap_exactOut() (gas: 168556)
[PASS] test_gas_MinRate_LimitSwap_quote_exactIn() (gas: 123591)
[PASS] test_gas_MinRate_LimitSwap_quote_exactOut() (gas: 123815)
[PASS] test_gas_MinRate_LimitSwap_swap_exactIn() (gas: 181486)
[PASS] test_gas_MinRate_LimitSwap_swap_exactOut() (gas: 181926)
[PASS] test_gas_Salt_LimitSwap_quote_exactIn() (gas: 120701)
[PASS] test_gas_Salt_LimitSwap_swap_exactIn() (gas: 178266)
[PASS] test_gas_TWAP_LimitSwap_quote_exactIn() (gas: 136085)
[PASS] test_gas_TWAP_LimitSwap_quote_exactOut() (gas: 137746)
[PASS] test_gas_TWAP_LimitSwap_swap_exactIn() (gas: 256279)
[PASS] test_gas_TWAP_LimitSwap_swap_exactOut() (gas: 256751)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 13.31ms (12.75ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

```
Ran 9 tests for test/invariants/pegged/LargeAmounts.t.sol:LargeAmounts
[PASS] test_Pegged() (gas: 6154897)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7428475)
[PASS] test_PeggedFlatFeeIn() (gas: 6298902)
[PASS] test_PeggedFlatFeeOut() (gas: 3067745)
[PASS] test_PeggedMultipleFees() (gas: 3065004)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2681039)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2681682)
[PASS] test_PeggedProtocolFee() (gas: 6811604)
[PASS] test_PeggedProtocolFeeIn() (gas: 6922188)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 113.81ms (113.20ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
revertTo(uint256): replaced by `revertToState`
snapshot(): replaced by `snapshotState`
```

```
Ran 5 tests for test/invariants/ConcentrateXYCInvariants.t.sol:ConcentrateXYCInvariants
[PASS] test_ConcentrateDifferentRanges() (gas: 22228782)
[PASS] test_ConcentrateGrowLiquidity2D() (gas: 5587085)
[PASS] test_ConcentrateGrowLiquidityXD() (gas: 5694682)
[PASS] test_ConcentrateGrowPriceRange2D() (gas: 5052101)
[PASS] test_ConcentrateGrowPriceRangeXD() (gas: 5152420)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 102.95ms (101.76ms CPU time)
```

```

Ran 3 tests for test/MakerHooks.t.sol:MakerHooksTest
[PASS] test_HooksExecutionOrder() (gas: 1202191)
[PASS] test_HooksWithEmptyTakerData() (gas: 689085)
[PASS] test_MakerHooksWithTakerData() (gas: 1253056)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 5.02ms (3.70ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/pegged/HugeLiquidity.t.sol:HugeLiquidity
[PASS] test_Pegged() (gas: 6153866)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7427656)
[PASS] test_PeggedFlatFeeIn() (gas: 6298072)
[PASS] test_PeggedFlatFeeOut() (gas: 3066456)
[PASS] test_PeggedMultipleFees() (gas: 3064139)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2689703)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2690925)
[PASS] test_PeggedProtocolFee() (gas: 6811230)
[PASS] test_PeggedProtocolFeeIn() (gas: 6921351)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 127.42ms (126.07ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/xyz/ImbalancedPoolHighFees.t.sol:ImbalancedPoolHighFees
[PASS] test_XYC() (gas: 4970805)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6244773)
[PASS] test_XYCFatFeeIn() (gas: 5115005)
[PASS] test_XYCFatFeeOut() (gas: 2307514)
[PASS] test_XYCMultipleFees() (gas: 2445510)
[PASS] test_XYCProgressiveFeeIn() (gas: 2071499)
[PASS] test_XYCProgressiveFeeOut() (gas: 2072215)
[PASS] test_XYCProtocolFee() (gas: 5628086)
[PASS] test_XYCProtocolFeeIn() (gas: 5665228)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 98.10ms (96.86ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/pegged/VeryLinear.t.sol:VeryLinear
[PASS] test_Pegged() (gas: 6150958)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7424536)
[PASS] test_PeggedFlatFeeIn() (gas: 6294963)
[PASS] test_PeggedFlatFeeOut() (gas: 3062531)
[PASS] test_PeggedMultipleFees() (gas: 3059372)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2675407)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2676050)
[PASS] test_PeggedProtocolFee() (gas: 6807665)
[PASS] test_PeggedProtocolFeeIn() (gas: 6918249)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 127.10ms (126.47ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/XYCFeesInvariants.t.sol:XYCFeesInvariants
[PASS] test_XYC() (gas: 4985926)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6259897)
[PASS] test_XYCFatFeeIn() (gas: 5130126)
[PASS] test_XYCFatFeeOut() (gas: 2322001)
[PASS] test_XYCMultipleFees() (gas: 2459981)
[PASS] test_XYCProgressiveFeeIn() (gas: 2085969)
[PASS] test_XYCProgressiveFeeOut() (gas: 2086674)
[PASS] test_XYCProtocolFee() (gas: 5643179)
[PASS] test_XYCProtocolFeeIn() (gas: 5680349)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 76.68ms (76.10ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/xyz/ImbalancedPoolLowFees.t.sol:ImbalancedPoolLowFees
[PASS] test_XYC() (gas: 4960342)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6234309)
[PASS] test_XYCFatFeeIn() (gas: 5104542)
[PASS] test_XYCFatFeeOut() (gas: 2297109)
[PASS] test_XYCMultipleFees() (gas: 2435118)

```

```
[PASS] test_XYCProgressiveFeeIn() (gas: 2061107)
[PASS] test_XYCProgressiveFeeOut() (gas: 2061823)
[PASS] test_XYCProtocolFee() (gas: 5617524)
[PASS] test_XYCProtocolFeeIn() (gas: 5654765)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 72.74ms (72.12ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 9 tests for test/invariants/pegged/VeryImbalancedDifferentDecimals.t.sol:VeryImbalancedDifferentDecimals
[PASS] test_Pegged() (gas: 6112965)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7386537)
[PASS] test_PeggedFlatFeeIn() (gas: 6256969)
[PASS] test_PeggedFlatFeeOut() (gas: 3025712)
[PASS] test_PeggedMultipleFees() (gas: 3022759)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2650229)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2649849)
[PASS] test_PeggedProtocolFee() (gas: 6769669)
[PASS] test_PeggedProtocolFeeIn() (gas: 6880254)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 108.21ms (107.61ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/pegged/ImbalancedPoolHighFees.t.sol:ImbalancedPoolHighFees
[PASS] test_Pegged() (gas: 6125562)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7399137)
[PASS] test_PeggedFlatFeeIn() (gas: 6269356)
[PASS] test_PeggedFlatFeeOut() (gas: 3041546)
[PASS] test_PeggedMultipleFees() (gas: 3039628)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2666484)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2666926)
[PASS] test_PeggedProtocolFee() (gas: 6782238)
[PASS] test_PeggedProtocolFeeIn() (gas: 6892852)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 113.37ms (112.75ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  snapshot(): replaced by `snapshotState`
  revertTo(uint256): replaced by `revertToState`
Ran 9 tests for test/invariants/pegged/LargeDifferentDecimals.t.sol:LargeDifferentDecimals
[PASS] test_Pegged() (gas: 6127920)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7401775)
[PASS] test_PeggedFlatFeeIn() (gas: 6272120)
[PASS] test_PeggedFlatFeeOut() (gas: 3041219)
[PASS] test_PeggedMultipleFees() (gas: 3038943)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2664510)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2665732)
[PASS] test_PeggedProtocolFee() (gas: 6785280)
[PASS] test_PeggedProtocolFeeIn() (gas: 6895473)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 123.77ms (121.49ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 9 tests for test/invariants/xyz/LargeAmounts.t.sol:LargeAmounts
[PASS] test_XYC() (gas: 4992822)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6266793)
[PASS] test_XYCFatFeeIn() (gas: 5137023)
[PASS] test_XYCFatFeeOut() (gas: 2328882)
[PASS] test_XYCMultipleFees() (gas: 2466873)
[PASS] test_XYCProgressiveFeeIn() (gas: 2092860)
[PASS] test_XYCProgressiveFeeOut() (gas: 2093576)
[PASS] test_XYCProtocolFee() (gas: 5650105)
[PASS] test_XYCProtocolFeeIn() (gas: 5687247)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 84.36ms (85.30ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:
  revertTo(uint256): replaced by `revertToState`
  snapshot(): replaced by `snapshotState`
Ran 9 tests for test/invariants/xyz/TinyLiquidity.t.sol:TinyLiquidity
[PASS] test_XYC() (gas: 4985652)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6259622)
[PASS] test_XYCFatFeeIn() (gas: 5129852)
[PASS] test_XYCFatFeeOut() (gas: 2321823)
[PASS] test_XYCMultipleFees() (gas: 2459822)
```

```
[PASS] test_XYCProgressiveFeeIn() (gas: 2085810)
[PASS] test_XYCProgressiveFeeOut() (gas: 2086526)
[PASS] test_XYCProtocolFee() (gas: 5642876)
[PASS] test_XYCProtocolFeeIn() (gas: 5680075)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 62.26ms (59.20ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

Ran 9 tests for test/invariants/xyz/HugeLiquidity.t.sol:HugeLiquidity

```
[PASS] test_XYC() (gas: 4985941)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6259912)
[PASS] test_XYCFatFeeIn() (gas: 5130141)
[PASS] test_XYCFatFeeOut() (gas: 2321989)
[PASS] test_XYCMultipleFees() (gas: 2459981)
[PASS] test_XYCProgressiveFeeIn() (gas: 2085969)
[PASS] test_XYCProgressiveFeeOut() (gas: 2086685)
[PASS] test_XYCProtocolFee() (gas: 5643153)
[PASS] test_XYCProtocolFeeIn() (gas: 5680365)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 130.59ms (99.25ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

Ran 9 tests for test/invariants/xyz/SmallAmounts.t.sol:SmallAmounts

```
[PASS] test_XYC() (gas: 6478340)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 8030925)
[PASS] test_XYCFatFeeIn() (gas: 6655351)
[PASS] test_XYCFatFeeOut() (gas: 2910950)
[PASS] test_XYCMultipleFees() (gas: 3062962)
[PASS] test_XYCProgressiveFeeIn() (gas: 2595504)
[PASS] test_XYCProgressiveFeeOut() (gas: 2596372)
[PASS] test_XYCProtocolFee() (gas: 7332968)
[PASS] test_XYCProtocolFeeIn() (gas: 7383042)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 84.83ms (101.33ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

Ran 9 tests for test/invariants/xyz/BalancedPoolEdgeFees.t.sol:BalancedPoolEdgeFees

```
[PASS] test_XYC() (gas: 4992558)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 6266529)
[PASS] test_XYCFatFeeIn() (gas: 5104894)
[PASS] test_XYCFatFeeOut() (gas: 2328828)
[PASS] test_XYCMultipleFees() (gas: 2435059)
[PASS] test_XYCProgressiveFeeIn() (gas: 2092794)
[PASS] test_XYCProgressiveFeeOut() (gas: 2093510)
[PASS] test_XYCProtocolFee() (gas: 5649812)
[PASS] test_XYCProtocolFeeIn() (gas: 5686983)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 115.27ms (72.39ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`
```

Ran 8 tests for test/XYCConcentrate.t.sol:ConcentrateTest

```
[PASS] test_ConcentrateGrowLiquidity_ImpossibleSwapTokenNotInActiveStrategy() (gas: 1038478)
[PASS] test_ConcentrateGrowLiquidity_KeepsPriceRangeForBothTokensNoFee() (gas: 554343)
[PASS] test_ConcentrateGrowLiquidity_KeepsPriceRangeForBothTokensWithFee() (gas: 554184)
[PASS] test_ConcentrateGrowLiquidity_KeepsPriceRangeForTokenA() (gas: 365413)
[PASS] test_ConcentrateGrowLiquidity_KeepsPriceRangeForTokenB() (gas: 365546)
[PASS] test_ConcentrateGrowLiquidity_SpreadSlowlyGrowsForSomeReason() (gas: 26746181)
[PASS] test_QuoteAndSwapExactOutAmountsMatches() (gas: 330464)
[PASS] test_RoundingInvariantsWithFees() (gas: 39489126)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 186.36ms (185.61ms CPU time)
```

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

```
revertTo(uint256): replaced by `revertToState`
snapshot(): replaced by `snapshotState`
```

Ran 6 tests for test/XYCSwap.t.sol:XYCSwapTest

```
[PASS] test_XYCSwap_BasicSwap_NoFee() (gas: 218223)
[PASS] test_XYCSwap_BasicSwap_WithFee() (gas: 231657)
[PASS] test_XYCSwap_MultipleSwaps_UpdatesState() (gas: 282171)
[PASS] test_XYCSwap_RoundingInvariants_HighFee() (gas: 32070254)
[PASS] test_XYCSwap_RoundingInvariants_NoFee() (gas: 30435104)
[PASS] test_XYCSwap_RoundingInvariants_WithFee() (gas: 32070848)
```

Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 228.87ms (328.68ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

revertTo(uint256): replaced by `revertToState`
snapshot(): replaced by `snapshotState`

Ran 9 tests for test/invariants/pegged/ImbalancedPoolLowFees.t.sol:ImbalancedPoolLowFees

[PASS] test_Pegged() (gas: 6125562)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7398961)
[PASS] test_PeggedFlatFeeIn() (gas: 6269566)
[PASS] test_PeggedFlatFeeOut() (gas: 3041546)
[PASS] test_PeggedMultipleFees() (gas: 3039662)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2666484)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2666926)
[PASS] test_PeggedProtocolFee() (gas: 6782062)
[PASS] test_PeggedProtocolFeeIn() (gas: 6892676)

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 307.59ms (112.55ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

snapshot(): replaced by `snapshotState`
revertTo(uint256): replaced by `revertToState`

Ran 9 tests for test/invariants/xyz/DustAmounts.t.sol:DustAmounts

[PASS] test_XYC() (gas: 11884299)
[PASS] test_XYCDynamicProtocolFeeIn() (gas: 13908095)
[PASS] test_XYCFatFeeIn() (gas: 12191630)
[PASS] test_XYCFatFeeOut() (gas: 5172478)
[PASS] test_XYCMultipleFees() (gas: 5084628)
[PASS] test_XYCProgressiveFeeIn() (gas: 4457601)
[PASS] test_XYCProgressiveFeeOut() (gas: 4458809)
[PASS] test_XYCProtocolFee() (gas: 12910765)
[PASS] test_XYCProtocolFeeIn() (gas: 13006794)

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 357.94ms (170.66ms CPU time)

Ran 25 tests for test/XYCSwapAqua.t.sol:XYCSwapAquaTest

[PASS] test_Aqua_XYC_Dust_ExactIn_KInvariant() (gas: 441837)
[PASS] test_Aqua_XYC_Dust_ExactIn_KInvariant_Balanced() (gas: 334332)
[PASS] test_Aqua_XYC_Dust_ExactIn_SwapRateVsSpot() (gas: 436816)
[PASS] test_Aqua_XYC_Dust_ExactOut_KInvariant() (gas: 442654)
[PASS] test_Aqua_XYC_Dust_ExactOut_KInvariant_Balanced() (gas: 441496)
[PASS] test_Aqua_XYC_Dust_ExactOut_SwapRateVsSpot() (gas: 436466)
[PASS] test_Aqua_XYC_ExactIn_BalancesCorrect() (gas: 437533)
[PASS] test_Aqua_XYC_ExactIn_NoRoundingAccumulation() (gas: 104922828)
[PASS] test_Aqua_XYC_ExactIn_OverflowReverts() (gas: 330468)
[PASS] test_Aqua_XYC_ExactIn_PriceImpactGrows() (gas: 865591)
[PASS] test_Aqua_XYC_ExactIn_SingleBeatsSplit() (gas: 951504)
[PASS] test_Aqua_XYC_ExactOut_BalancesCorrect() (gas: 460163)
[PASS] test_Aqua_XYC_ExactOut_NoRoundingAccumulation() (gas: 105250407)
[PASS] test_Aqua_XYC_ExactOut_OverflowReverts() (gas: 331108)
[PASS] test_Aqua_XYC_ExactOut_PriceImpactGrows() (gas: 867229)
[PASS] test_Aqua_XYC_ExactOut_SingleBeatsSplit() (gas: 954033)
[PASS] test_Aqua_XYC_MaxAmount_ExactIn_KInvariant() (gas: 441438)
[PASS] test_Aqua_XYC_MaxAmount_ExactIn_SwapRateVsSpot() (gas: 435691)
[PASS] test_Aqua_XYC_MaxAmount_ExactOut_KInvariant() (gas: 441717)
[PASS] test_Aqua_XYC_MaxAmount_ExactOut_SwapRateVsSpot() (gas: 436812)
[PASS] test_Aqua_XYC_NoArbitrage_InThenIn() (gas: 545896)
[PASS] test_Aqua_XYC_NoArbitrage_InThenOut() (gas: 553118)
[PASS] test_Aqua_XYC_NoArbitrage_OutThenIn() (gas: 547387)
[PASS] test_Aqua_XYC_NoArbitrage_OutThenOut() (gas: 551709)
[PASS] test_Aqua_XYC_RoundingFavorsProtocol() (gas: 480087)

Suite result: ok. 25 passed; 0 failed; 0 skipped; finished in 436.69ms (559.91ms CPU time)

Warning: the following cheatcode(s) are deprecated and will be removed in future versions:

revertTo(uint256): replaced by `revertToState`
snapshot(): replaced by `snapshotState`

Ran 9 tests for test/invariants/pegged/SmallAmounts.t.sol:SmallAmounts

[PASS] test_Pegged() (gas: 6150950)
[PASS] test_PeggedDynamicProtocolFeeIn() (gas: 7424528)
[PASS] test_PeggedFlatFeeIn() (gas: 6294955)
[PASS] test_PeggedFlatFeeOut() (gas: 3062525)
[PASS] test_PeggedMultipleFees() (gas: 3059366)
[PASS] test_PeggedProgressiveFeeIn() (gas: 2675401)
[PASS] test_PeggedProgressiveFeeOut() (gas: 2676044)
[PASS] test_PeggedProtocolFee() (gas: 6807657)
[PASS] test_PeggedProtocolFeeIn() (gas: 6918241)

Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 487.58ms (116.84ms CPU time)

```
Ran 65 test suites in 682.05ms (6.01s CPU time): 591 tests passed, 0 failed, 0 skipped (591 total tests)
```

8.1.3 solidity-utils

[illegible]

```

CalldataPtr
  from
    should create CalldataPtr from bytes (38ms)
    should create CalldataPtr from empty bytes
  getOffsetAndLength
    should return correct length for data
    should return correct length for empty data
    should return non-zero offset
  roundTrip
    should return same data after from and toBytes
    should return empty for empty input
    should preserve arbitrary data

BytesMemoryMock
  wrap
    should return correct pointer and length of data (2051ms)
    should return correct pointer and length of empty data
    should return correct pointer and length of data with non-default pointer
  slice
    should revert with incorrect offset
    should revert with incorrect size
    should revert with incorrect offset + size
    should slice data
  unwrap
    should return correct bytes after wrap
    should return correct bytes after wrap with non-default pointer
    should return correct bytes slice
  Gas usage
    unwrap 32 bytes
    unwrap 33 bytes
    unwrap 64 bytes
    slice

```

```

EthReceiver
  contract transfer
  normal transfer

```

```

(node:72858) PromiseRejectionHandledWarning: Promise rejection was handled asynchronously (rejection id: 3)
(Use `node --trace-warnings ...` to show where the warning was created)
(node:72860) PromiseRejectionHandledWarning: Promise rejection was handled asynchronously (rejection id: 3)
(Use `node --trace-warnings ...` to show where the warning was created)

```

```

BytesStorageMock
  wrap
    should return correct wrapped data with short data (2202ms)
    should return correct wrapped data with long data (43ms)
  slice
    short data
      should revert with incorrect offset (128ms)
      should revert with incorrect size
      should revert with incorrect offset + size
      should slice data
    long data
      should revert with incorrect offset with long data (46ms)
      should revert with incorrect size with long data
      should revert with incorrect offset + size with long data
      should slice data
  copy
    short data
      should return the same bytes after copy
      should return correct bytes after slice and copy
    long data
      should return the same bytes after copy
      should return correct bytes after slice and copy
      should return correct bytes after slice and copy more than 50 bytes

BySig
  bySigAccountNonces and useBySigAccountNonce
    should return current nonce and correct change it (2238ms)
  bySigSelectorNonces and useBySigSelectorNonce
    should return current nonce and correct change it
  bySigUniqueNonces and useBySigUniqueNonce and bySigUniqueNoncesSlot

```

```

    should return true if nonce equals to setted nonce, false in another case and correct change it (150ms)
hashBySig
    should return correct hash
bySig
    should return false for an invalid nonce type
    should revert after traits deadline
    should revert if relayer denied
    should revert with wrong Account nonce
    should revert with wrong Selector nonce
    should revert with wrong Unique nonce
    should revert with wrong signature when no data
    should revert with wrong signature
    should work for transfer method
    should make approve for sponsored call
    should work recursively
    should work recursively for sponsored call

```

Multicall

```

multicall
    should execute single call (67ms)
    should execute multiple calls
    should execute empty array
    should preserve msg.sender in delegatecall
    should revert on failed call and forward error
    should revert on failed call in middle of batch
    should forward empty revert

```

RevertReasonForwarder

```

    should forward custom error from the helper contract using reRevert
    should return custom error from the helper contract using reReason

```

Permitable

```

    should be permitted for IERC20Permit (287ms)
    should be permitted for IERC20Permit (compact)
    should be permitted for IERC20Permit with deadline less than max int (46ms)
    should be not permitted for IERC20Permit with deadline less than current block (41ms)
    should not be permitted for IERC20Permit
    should be permitted for IDaiLikePermit
    should be permitted for IPermit2
    should be permitted for IPermit2, compact
    should be permitted for IDaiLikePermit (compact)
    should not be permitted for IDaiLikePermit
    should be permitted for IERC7597Permit

```

RevertReasonParser

```

    should be parsed as Unknown (Invalid revert reason)
    should be parsed as empty Error
    should be parsed as Error
    should be parsed as Unknown
    should be parsed as Panic
    should be parsed as Error with long string
    should be reverted in _test()

```

Permitable

```

    should work with valid permit (66ms)
    should work with invalid permit
    should work with nested permit
    should work with payable function
    should work with payable function and nested permit

```

ReentrancyGuard

```

nonReentrant modifier
    should allow normal function call (43ms)
    should allow multiple sequential calls
    should prevent reentrancy attack
onlyNonReentrantCall modifier
    should allow call within nonReentrant context
    should revert when called outside nonReentrant context
_inNonReentrantCall

```

```

    should return false when not in protected context
    should return true when in protected context
  transient behavior
    should reset lock between transactions
  nonReentrantLock modifier (custom lock)
    should allow normal function call with custom lock
    should allow multiple sequential calls with custom lock
    should prevent reentrancy attack with custom lock
    should have independent locks (custom vs built-in)
  onlyNonReentrantCallLock modifier (custom lock)
    should allow call within custom lock context
    should revert when called outside custom lock context

send ethers without selfdestruct 30962
send all ethers 29185

SelfdestructEthSender
  should send Ethers with selfdestruct (46ms)

(node:72860) PromiseRejectionHandledWarning: Promise rejection was handled asynchronously (rejection id: 4)

AddressSet
  length
    should get length 0 (63ms)
    should get length 1
  at
    should not get from empty set
    should not get index out of array length
    should get from set with 1 element
    should get from set with several elements
  unsafeAt
    should get from empty set
    should get from set with 1 element
    should get from set with several elements
  get
    should get empty array
    should get array with 1 element
    should get array with 2 elements
    should get from array with 3 elements
    should get array with 2 elements and copies the addresses into the provided input array
    should reverted because provided input array size is too small
  contains
    should not contain in empty set
    should contain 1 address
    should contains several addresses
  add
    should add to empty set
    should not add element twice
    should add to set with 1 element
    should not add element twice to set with 1 element
  remove
    should not remove from empty set
    should remove from set
    should not remove element which is not in set
    should remove from set and keep other elements
  multiple add/remove
    should add and remove multiple times
  erase
    should not change empty array
    should reset non-zero array length
    should reset non-zero array
    should not return item from array after reset

(node:72858) PromiseRejectionHandledWarning: Promise rejection was handled asynchronously (rejection id: 4)

AddressArray
  length
    should calculate length 0 (77ms)
    should calculate length 1
  at
    should not get from empty array
    should not get index out of array length
    should get from array with 1 element
    should get from array with several elements
  unsafeAt

```

```

    should get from empty array
    should get from array with 1 element
    should get from array with several elements (52ms)
  get
    should get empty array
    should get array with 1 element
    should get array with 2 elements
    should get from array with 3 elements
    should get array with 2 elements and copies the addresses into the provided input array
    should revert because provided input array size is too small
  push
    should push to empty array
    should push to array with 1 element
    should push to array with 2 elements
  pop
    should throw when array is empty
    should pop from array with 1 element
    should pop from array with 2 elements
    should pop from array with 3 elements
    should throw when pops more than there are elements in array
  set
    should throw when sets index out of bounds
    should set index 0 in array with 1 element
    should set index 0 in array with several elements
    should set index 1 in array with several elements
  multiple add/remove
    should add and remove multiple times
  erase
    should not change empty array
    should reset non-zero array length
    should reset non-zero array (53ms)
    should not return item from array after reset

```

Simulator

```

  simulate
    should always revert with Simulated error
    should return success=true for successful delegatecall
    should return success=false for failing delegatecall
    should not persist state changes
    should accept ETH value

```

WethReceiver

```

  contract transfer
  normal transfer

```

TransientLock

```

  lock
    should lock successfully when unlocked (46ms)
    should be locked after lock() call within same transaction
  unlock
    should revert when unlocking without lock
  isLocked
    should return false initially
    should return false after lock and unlock in same transaction
  double lock
    should revert on double lock
  transient behavior
    should reset lock state between transactions

```

Transient

```

  tuint256
    tload/tstore
      should return 0 for uninitialized value
      should store and load value in same tx via staticCall
      should clear value between transactions
    inc
      should increment from 0 to 1
      should increment sequentially in same transaction
      should revert on overflow (when incremented == 0)
      should revert with custom exception on overflow
    dec

```

```

    should revert on underflow from 0
    should revert with custom exception on underflow
unsafeInc/unsafeDec
    should increment without check
    should underflow without reverting
initAndAdd
    should initialize with value if zero and add
taddress
    should return zero address for uninitialized
    should clear address between transactions
tbytes32
    should return zero for uninitialized
    should clear bytes32 between transactions

```

Permit library

```

    should be trimmed
    should not be changed
    should correctly build data for permit (38ms)
    should correctly build data for dai-like permit
    should concat target with prefixed data
    should concat target with raw data

```

trace inspection

```

profileEVM
    should be counted ERC20 Transfer
    should be counted ERC20 Approve
gasspectEVM
    should be counted ERC20 Transfer
    should be counted ERC20 Approve
    should be counted ERC20 Transfer with minOpGasCost = 2000
    should be counted ERC20 Transfer with args
    should be counted ERC20 Transfer with res

```

UniERC20

```

ERC20 token
    is ETH
    uni transfer
    uni transfer from
    uni approve
ERC20 NAME/SYMBOL
    uni name
    uni symbol
ERC20 token, empty name/symbol
    uni name
    uni symbol
ERC20 NAME/SYMBOL bytes32
    uni name
    uni symbol
ERC20 name/symbol bytes32
    uni name
    uni symbol
ERC20 without name/symbol
    uni name
    uni symbol
ETH
    is ETH
    uni transfer
    uni transfer, msg.value > amount
    uni transfer, insufficient balance
    uni approve must fail
    uni transfer from, success
    uni transfer from, fail, not sender
    uni transfer from, fail, receiver is not contract
    uni name
    uni symbol
ETH with bad ether receiver
    uni failed transfer
    uni failed transferFrom
ETH from special address
    is ETH

```

```
balanceOf:safeBalanceOf gasUsed - 26955:26807
```

```

StringUtil
Validity
  Uint 256
  Uint 128
  Very long byte array
  Extremely long byte array
  Empty bytes
  Single byte
  Random bytes
  Same bytes short
  Same bytes long
Gas usage
  Uint 256
  Uint 256 naive
  Uint 256 as bytes
  Uint 256 as bytes naive
  Uint 128
  Uint 128 naive
  Very long byte array gas
  Very long byte array gas naive
  Extremely long byte array gas
  Extremely long byte array gas naive
  Empty bytes
  Empty bytes naive
  Single byte
  Single byte naive
  Compare gas usage Uint256
  Compare gas usage Uint256 as bytes
  Compare gas usage Uint128
  Compare gas usage very long byte array
  Compare gas usage extremely long byte array
  Compare gas usage empty bytes
  Compare gas usage single byte

ECDSA
recover
  with invalid signature
    with short signature (48ms)
    with long signature
  with valid signature
    using account.signMessage
      returns signer address with correct signature
      returns signer address with correct signature for arbitrary length message
      returns a different address
      returns zero address with invalid signature
    with v0 signature
      returns zero address with 00 as version value
      works with 27 as version value
      returns zero address when wrong version
      works with short EIP2098 format
    with v1 signature
      returns zero address with 01 as version value
      works with 28 as version value
      returns zero address when wrong version
      works with short EIP2098 format
isValidSignature
  with invalid signature
    with short signature
    with long signature
  with valid signature
    using account.signMessage
      returns true with correct signature and only correct signer
      returns true with correct signature and only correct signer for arbitrary length message
      returns false with invalid signature
    with v0 signature
      returns false with 00 as version value
      returns true with 27 as version value, and only for signer
      returns false when wrong version
      returns true with short EIP2098 format, and only for signer
    with v1 signature
      returns false with 01 as version value
      returns true with 28 as version value, and only for signer
      returns false when wrong version

```

```

        returns true with short EIP2098 format, and only for signer
    isValidSignature65
        with matching signer and signature
        with invalid signer
        with invalid signature
    recoverOrIsValidSignature
        with invalid signature
        with short signature
        with long signature
    with valid signature
        using account.signMessage
            returns true with correct signature and only correct signer
            returns true with correct signature and only correct signer for arbitrary length message
            returns false with invalid signature
        with v0 signature
            returns false with 00 as version value
            returns true with 27 as version value, and only for signer
            returns false when wrong version
            returns true with short EIP2098 format, and only for signer
        with v1 signature
            returns false with 01 as version value
            returns true with 28 as version value, and only for signer
            returns false when wrong version
            returns true with short EIP2098 format, and only for signer
    recoverOrIsValidSignature65
        with matching signer and signature
        with invalid signer
        with invalid signature
    toEthSignedMessageHash
        correct hash
    toTypedDataHash
        correct hash
    gas price
    recover
        with signature
        with v0 signature
        with v1 signature
    recoverOrIsValidSignature
        with signature
        with v0 signature
        with v1 signature
    recoverOrIsValidSignature65
    isValidSignature
        with signature
        with v0 signature
        with v1 signature
    isValidSignature65
    Additional methods
        toEthSignedMessageHash
        toTypedDataHash

```

SafeERC20

```

    with address that has no contract code
        reverts on transfer
        reverts on transferFrom
        reverts on approve
        reverts on increaseAllowance
        reverts on decreaseAllowance
    with token that returns false on all calls
        reverts on transfer
        reverts on transferFrom
        reverts on approve
        reverts on increaseAllowance
        reverts on decreaseAllowance
    with token that returns true on all calls
        doesn't revert on transfer
        doesn't revert on transferFrom
        doesn't revert on transferFromUniversal, permit2
        doesn't revert on transferFromUniversal, no permit2
    approvals
        with zero allowance
            doesn't revert when approving a non-zero allowance
            doesn't revert when approving a zero allowance
            doesn't revert when increasing the allowance

```

```

    reverts when decreasing the allowance
with non-zero allowance
    doesn't revert when approving a non-zero allowance
    doesn't revert when approving a zero allowance
    doesn't revert when increasing the allowance
    doesn't revert when decreasing the allowance to a positive value
    reverts when decreasing the allowance to a negative value
with token that returns no boolean values
    doesn't revert on transfer
    doesn't revert on transferFrom
    doesn't revert on transferFromUniversal, permit2
    doesn't revert on transferFromUniversal, no permit2
approvals
    with zero allowance
        doesn't revert when approving a non-zero allowance
        doesn't revert when approving a zero allowance
        doesn't revert when increasing the allowance
        reverts when decreasing the allowance
    with non-zero allowance
        doesn't revert when approving a non-zero allowance
        doesn't revert when approving a zero allowance
        doesn't revert when increasing the allowance
        doesn't revert when decreasing the allowance to a positive value
        reverts when decreasing the allowance to a negative value
non-zero to non-zero approval forbidden
    zero to non-zero approval should pass
    non-zero to non-zero approval should pass
    non-zero to non-zero approval should revert if approve failed
    non-zero to zero to non-zero approval should pass
safeBalanceOf
    should return zero balance
    should return balance
    should be cheaper than balanceOf
with token that doesn't revert on invalid permit
    accepts owner signature
    revert on reused signature
    revert on invalid signature
IWETH methods
    should deposit tokens
    should withdrawal tokens on withdraw
    should withdrawal tokens on withdrawTo
    should be cheap on withdrawTo to self

Skipping verification
TokenMock deployed to: 0x9B0520cC84E43E9D04f3DB36019c27Da33f71Fc3
Skipping verification
TokenMock deployed to: 0x48734CDc398c9f8aC06BAd6B53E58e54101E3378
Skipping verification
Contract TokenMock is already deployed at 0x48734CDc398c9f8aC06BAd6B53E58e54101E3378
TokenMock deployed to: 0x35b85954e2640c136d8D70E676749DFbF4b9c4FE
Skipping verification
Skipping verification

timeIncreaseTo
    should be increased on 1000 sec (670ms)
    should be increased on 2000 sec (1000ms)
    should be increased on 1000000 sec (999ms)
    should be thrown with increase time to a moment in the past (1010ms)

fixSignature
    should not be fixed geth sign
    should be fixed ganache sign

utils
    signMessage
        should be signed 0x message
        should be signed 32 bytes random bytes
        should be signed string -> Uint8Array -> hex string -> Uint8Array
        should be signed string -> Uint8Array -> hex string
    trackReceivedTokenAndTx
        should be tracked ERC20 Transfer
        should be tracked ERC20 Approve
    trackReceivedToken
        should be tracked ERC20 Transfer
        should be tracked ERC20 Approve

```

```
countInstructions
  should be counted ERC20 Transfer
  should be counted ERC20 Approve
deployContract
  should be deploy new contract instance
  should be using without arguments
deployContractFromBytecode
  should deploy new contract instance
  can be used without arguments
deployAndGetContract
  should deploy new contract instance (42ms)
deployAndGetContractWithCreate3
  should deploy new contract instance to the pre-known address
  should skip if set skipIfAlreadyDeployed
saveContractWithCreate3Deployment
  should save deployment by tx hash
getEthPrice
  should return ETH price (581ms)
  should return BNB price (473ms)
  should throw error with incorrect token symbol (454ms)
getAccountsWithCode
  should return accounts without bytecode by default
  should return accounts with the same specific bytecode
  should return specific accounts with specific bytecode
```

441 passing (12s)

Done in 23.89s.

8.2 Automated Tools

8.2.1 AuditAgent

The AuditAgent is an AI-powered smart contract auditing tool that analyses code, detects vulnerabilities, and provides actionable fixes. It accelerates the security analysis process, complementing human expertise with advanced AI models to deliver efficient and comprehensive smart contract audits. Available at <https://app.auditagent.nethermind.io>.

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to [Nethermind](#) in order that [Nethermind](#) could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. [Nethermind](#) has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, [Nethermind](#) disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. [Nethermind](#) does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and [Nethermind](#) will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.